

of v_1 and v_2 implies that $v_2 \notin \text{span}\{v_1\} = \text{span}\{q_1\}$. Let $r_{22} = \|\hat{q}_2\|_2 \neq 0$, and define $q_2 = (1/r_{22})\hat{q}_2$. Then clearly $\|q_2\|_2 = 1$, and $\langle q_1, q_2 \rangle = 0$. It is intuitively clear from Figure 3.5 that $\text{span}\{q_1, q_2\} = \text{span}\{v_1, v_2\}$, and we can easily prove that it is so. First $q_1 \in \text{span}\{v_1\} \subseteq \text{span}\{v_1, v_2\}$. Also $q_2 \in \text{span}\{q_1, v_2\} = \text{span}\{v_1, v_2\}$. Since $q_1, q_2 \in \text{span}\{v_1, v_2\}$, we have $\text{span}\{q_1, q_2\} \subseteq \text{span}\{v_1, v_2\}$. Conversely, $v_1 \in \text{span}\{q_1\} \subseteq \text{span}\{q_1, q_2\}$, and $v_2 = r_{12}q_1 + \hat{q}_2 = r_{12}q_1 + r_{22}q_2 \in \text{span}\{q_1, q_2\}$. Since $v_1, v_2 \in \text{span}\{q_1, q_2\}$, we have $\text{span}\{v_1, v_2\} \subseteq \text{span}\{q_1, q_2\}$. Thus $\text{span}\{q_1, q_2\} = \text{span}\{v_1, v_2\}$.

Now consider the k th step. Suppose we have found orthonormal vectors $q_1, q_2, \dots, q_{k-1}$ such that $\text{span}\{q_1, \dots, q_i\} = \text{span}\{v_1, \dots, v_i\}$, $i = 1, \dots, k-1$. Let us see how to determine q_k . By analogy with (3.4.13) we seek $\hat{q}_k$ of the form

$$\hat{q}_k = v_k - \sum_{j=1}^{k-1} r_{jk} q_j \quad (3.4.15)$$

that is orthogonal to $q_1, \dots, q_{k-1}$. The equations $\langle \hat{q}_k, q_i \rangle = 0$, $i = 1, \dots, k-1$, imply that

$$\langle v_k, q_i \rangle - \sum_{j=1}^{k-1} r_{jk} \langle q_j, q_i \rangle = 0 \quad i = 1, \dots, k-1.$$

Since $\langle q_j, q_i \rangle = 0$ when $i \neq j$, and $\langle q_i, q_i \rangle = 1$, these equations reduce to

$$r_{ik} = \langle v_k, q_i \rangle \quad i = 1, \dots, k-1. \quad (3.4.16)$$

Conversely, if r_{ik} are defined by (3.4.16), then $\hat{q}_k$ is orthogonal to $q_1, \dots, q_{k-1}$.

As in the case $k = 2$, it is easy to show that $\hat{q}_k \neq 0$. Otherwise we would have $v_k = \sum_{j=1}^{k-1} r_{jk} q_j \in \text{span}\{q_1, \dots, q_{k-1}\}$. But the linear independence of $v_1, \dots, v_k$ implies that $v_k \notin \text{span}\{v_1, \dots, v_{k-1}\} = \text{span}\{q_1, \dots, q_{k-1}\}$. Let

$$r_{kk} = \|\hat{q}_k\|_2 > 0, \quad (3.4.17)$$

and define

$$q_k = \frac{1}{r_{kk}} \hat{q}_k. \quad (3.4.18)$$

Then clearly $\|q_k\|_2 = 1$ and $\langle q_k, q_i \rangle = 0$, $i = 1, \dots, k-1$.

It is also easy to verify that $\text{span}\{q_1, \dots, q_k\} = \text{span}\{v_1, \dots, v_k\}$. See Exercises 3.4.38 and 3.4.39.

Equations 3.4.16, 3.4.15, 3.4.17, and 3.4.18 express the k th step of the classical Gram-Schmidt process. Performing this step for $k = 1, \dots, m$ produces $q_1, \dots, q_m$. The algorithm is summarized below.

Classical Gram-Schmidt Algorithm. Given linearly independent $v_1, v_2, \dots, v_m$, this algorithm produces orthonormal $q_1, q_2, \dots, q_m$ such that $\text{span}\{q_1, \dots, q_i\} = \text{span}\{v_1, \dots, v_i\}$ for $i = 1, \dots, m$. $q_1, \dots, q_m$ are stored over $v_1, \dots, v_m$.

$$\begin{aligned} & \text{for } k = 1, \dots, m \\ & \quad \left[\begin{array}{l} \text{for } i = 1, \dots, k-1 \quad (\text{skipped when } k = 1) \\ \quad \left[r_{ik} \leftarrow \langle v_k, v_i \rangle \quad (v_i \text{ is really } q_i.) \right. \\ \text{for } i = 1, \dots, k-1 \\ \quad \left[v_k \leftarrow v_k - v_i r_{ik} \right. \\ \quad (v_k \text{ now contains } \hat{q}_k.) \\ r_{kk} \leftarrow \|v_k\|_2 \\ \text{if } (r_{kk} = 0) \text{ set flag } (v_1, \dots, v_k \text{ are dependent}), \text{ exit} \\ v_k \leftarrow (1/r_{kk})v_k \end{array} \right. \end{array} \quad (3.4.19)$$

Gram-Schmidt equals QR

The promised connection with the QR decomposition follows directly from the Gram-Schmidt equations. Combining (3.4.15) and (3.4.18), and solving for v_k , we obtain

$$v_k = \sum_{j=1}^{k-1} r_{jk} q_j + r_{kk} q_k, \quad k = 1, \dots, m, \quad (3.4.20)$$

or

$$\begin{aligned} v_1 &= q_1 r_{11} \\ v_2 &= q_1 r_{12} + q_2 r_{22} \\ v_3 &= q_1 r_{13} + q_2 r_{23} + q_3 r_{33} \\ &\vdots \\ v_m &= q_1 r_{1m} + q_2 r_{2m} + \dots + q_m r_{mm}. \end{aligned}$$

These can be packed into a single matrix equation

$$\begin{bmatrix} v_1 & v_2 & \dots & v_m \end{bmatrix} = \begin{bmatrix} q_1 & q_2 & \dots & q_m \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & \dots & r_{1m} \\ 0 & r_{22} & \dots & r_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & r_{mm} \end{bmatrix}.$$

Defining

$$V = \begin{bmatrix} v_1 & v_2 & \dots & v_m \end{bmatrix} \in \mathbb{R}^{n \times m}$$

$$Q = \begin{bmatrix} q_1 & q_2 & \dots & q_m \end{bmatrix} \in \mathbb{R}^{n \times m}$$

$$R = \begin{bmatrix} r_{11} & r_{12} & \dots & r_{1m} \\ 0 & r_{22} & \dots & r_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & r_{mm} \end{bmatrix} \in \mathbb{R}^{m \times m}, \quad (3.4.21)$$

we see that V has full rank, Q is isometric, R is upper triangular with positive entries on the main diagonal (by (3.4.17)), and

$$V = QR.$$

Thus Q and R are the unique factors of V guaranteed by Theorem 3.4.8.

This proves that the Gram-Schmidt process provides another means of calculating a QR decomposition: Given a matrix $V \in \mathbb{R}^{n \times m}$ ($n \geq m$) of full rank, let $v_1, \dots, v_m$ denote the columns of V . Carry out the Gram-Schmidt procedure on $v_1, \dots, v_m$ to produce an orthonormal set $q_1, \dots, q_m$ and the coefficients r_{jk} . Then define Q and R by (3.4.21) to get the QR decomposition of V . Conversely, any method for computing the QR decomposition can be used to orthogonalize vectors: Let $v_1, \dots, v_m$ denote a linearly independent set that is to be orthonormalized. Define $V = [v_1 \cdots v_m] \in \mathbb{R}^{n \times m}$ and use reflectors, rotators, or any other means to produce the unique QR decomposition of V guaranteed by Theorem 3.4.8. Let $q_1, \dots, q_m$ denote the columns of the resulting isometry Q . Then by the uniqueness of the QR decomposition, $q_1, \dots, q_m$ are exactly the vectors that would be produced by the Gram-Schmidt process. Thus we have the main result of this section: The Gram-Schmidt orthogonalization is the same as the QR decomposition.

Exercise 3.4.22

- (a) Let $v_1 = [3, -3, 3, -3]^T$, $v_2 = [1, 2, 3, 4]^T$, and $S = \text{span}\{v_1, v_2\} \in \mathbb{R}^4$. Apply the Gram-Schmidt process to S to obtain an orthonormal basis of S . Save the coefficients r_{jk} .

- (b) Let

$$V = \begin{bmatrix} 3 & 1 \\ -3 & 2 \\ 3 & 3 \\ -3 & 4 \end{bmatrix} \in \mathbb{R}^{4 \times 2}.$$

Use the result of part (a) to build an isometric $Q \in \mathbb{R}^{4 \times 2}$ and an upper triangular $R \in \mathbb{R}^{2 \times 2}$ such that $V = QR$.

□

The Modified Gram-Schmidt Process

Since the Gram-Schmidt process yields a QR decomposition, it can, in principle, be used to solve least squares problems. Unfortunately the classical Gram-Schmidt process turns out to be numerically unstable; small roundoff errors can sometimes cause the computed vectors to be far from orthogonal. Perhaps surprisingly, a slight modification of the algorithm suffices to make it stable.

In the computation of $\hat{q}_k$, the classical Gram-Schmidt process calculates all of the coefficients r_{ik} at once, then makes the update

$$\hat{q}_k = v_k - \sum_{i=1}^{k-1} r_{ik} q_i$$

all at once. The *modified Gram-Schmidt process* computes the coefficients one at a time. As soon as $r_{1k} = \langle v_k, q_1 \rangle$ has been computed, it is used to update v_k :

$$v_k^{(1)} = v_k - r_{1k} q_1.$$

Now $v_k^{(1)}$ is orthogonal to q_1 . Next the coefficient r_{2k} is computed using $v_k^{(1)}$ instead of v_k . That is, we take $r_{2k} = \langle v_k^{(1)}, q_2 \rangle$. Then we do another update:

$$v_k^{(2)} = v_k^{(1)} - r_{2k} q_2$$

to get a vector $v_k^{(2)}$ that is orthogonal to both q_1 and q_2 . Now $v_k^{(2)}$ is used instead of v_k to compute r_{3k} , and so on. After $k-1$ such updates, we have $v_k^{(k-1)} (= \hat{q}_k)$, which is orthogonal to $q_1, \dots, q_{k-1}$. We then obtain q_k by normalizing $v_k^{(k-1)}$. Thus the k th step of the modified Gram-Schmidt process looks like this:

$$\begin{aligned} v_k^{(0)} &\leftarrow v_k \\ \text{for } i &= 1, \dots, k-1 \\ &\left[\begin{array}{l} \tilde{r}_{ik} = \langle v_k, \tilde{q}_i \rangle \\ v_k^{(i)} \leftarrow v_k^{(i-1)} - \tilde{r}_{ik} \tilde{q}_i \end{array} \right. \\ \tilde{r}_{kk} &\leftarrow \|v_k^{(k-1)}\|_2 \\ \tilde{q}_k &\leftarrow (1/\tilde{r}_{kk})v_k^{(k-1)} \end{aligned} \quad (3.4.23)$$

We have placed tildes on the q_i and r_{ik} because it is (perhaps) not immediately clear that the vectors and coefficients produced by this algorithm are (in exact arithmetic) the same as those produced by the classical Gram-Schmidt process. In Exercises 3.4.40 and 3.4.41 you will show that, in the absence of roundoff errors, the modified and classical Gram-Schmidt processes produce exactly the same output.

Exercise 3.4.24 This exercise refers to (3.4.23). Assume that $\tilde{q}_1, \dots, \tilde{q}_{k-1}$ are orthonormal to begin with.

- Show that $v_k^{(1)}$ is orthogonal to $\tilde{q}_1$.
- Show that $v_k^{(2)}$ is orthogonal to both $\tilde{q}_1$ and $\tilde{q}_2$.
- Prove by induction on j that $v_k^{(j)}$ is orthogonal to $\tilde{q}_1, \tilde{q}_2, \dots, \tilde{q}_j$.
- Prove that $\tilde{q}_1, \dots, \tilde{q}_{k-1}, \tilde{q}_k$ is an orthonormal set.

Therefore, by induction on k , the modified Gram-Schmidt process produces an orthonormal set. $\square$

When the modified Gram-Schmidt process is implemented as a computer program, all of the vectors $v_k^{(j)}$, $j = 1, \dots, k-1$, as well as the beginning and ending vectors v_k and q_k , can be stored in the same place. The algorithm then looks as follows.

Modified Gram-Schmidt Algorithm. Given linearly independent $v_1, v_2, \dots, v_m$, this algorithm produces orthonormal $q_1, q_2, \dots, q_m$ such that $\text{span}\{q_1, \dots, q_i\} = \text{span}\{v_1, \dots, v_i\}$ for $i = 1, \dots, m$. $q_1, \dots, q_m$ are stored over $v_1, \dots, v_m$.

$$\begin{aligned} &\text{for } k = 1, \dots, m \\ &\quad \left[\begin{array}{l} \text{for } i = 1, \dots, k-1 \quad (\text{skipped when } k = 1) \\ \quad \left[\begin{array}{l} r_{ik} \leftarrow \langle v_k, v_i \rangle \quad (v_i \text{ is really } q_i) \\ v_k \leftarrow v_k - v_i r_{ik} \end{array} \right. \\ \quad (v_k \text{ now contains } v_k^{(k-1)}) \\ \quad r_{kk} \leftarrow \|v_k\|_2 \\ \quad \text{if } (r_{kk} = 0) \text{ set flag } (v_1, \dots, v_k \text{ are dependent}), \text{ exit} \\ \quad v_k \leftarrow (1/r_{kk})v_k \end{array} \right. \end{aligned} \quad (3.4.25)$$

Comparing this with the Classical Gram-Schmidt algorithm (3.4.19), we see that the only difference is that the two i loops in (3.4.19) have been condensed to a single loop in (3.4.25). It is thus obvious that the two algorithms have exactly the same flop count.

The classical and modified Gram-Schmidt algorithms produce the same output in principle but behave differently in practice because they compute the coefficients r_{ik} differently:

$$r_{ik} = \langle v_k, q_i \rangle \quad \text{and} \quad \tilde{r}_{ik} = \langle v_k^{(i-1)}, q_i \rangle.$$

The two methods are identical up through the computation of q_2 , so we have to go at least to q_3 to see the difference. To get q_3 , we first compute

$$\hat{q}_3 = v_3 - r_{13}q_1 - r_{23}q_2.$$

We shall call $r_{23}q_2$ the *component of v_3 in the direction of q_2* . This is exactly the multiple of q_2 which when subtracted from v_3 yields a vector that is orthogonal to q_2 . In order to remove this component successfully, we need to compute the coefficient r_{23} accurately. In principle v_3 and $v_3^{(1)} = v_3 - r_{13}q_1$ have the same component in the direction of q_2 , and it makes no difference whether we compute $r_{23} = \langle v_3, q_2 \rangle$ or $\tilde{r}_{23} = \langle v_3^{(2)}, q_2 \rangle$. This is so because q_1 is orthogonal to q_2 ; the extra term $r_{13}q_1$ should (in theory) contribute nothing to the computation of the coefficient. However, in practice the vectors q_1 and q_2 are not exactly orthogonal, because of roundoff errors, so the term $r_{13}q_1$ does have a slight component in the direction of q_2 . Furthermore, roundoff errors are made in the subtraction of $r_{13}q_1$ from v_3 . The vector of roundoff errors also has a slight component in the direction of q_2 . The classical Gram-Schmidt process is oblivious to these errors. In contrast, the modified process, by working with $v_3^{(1)}$ instead of v_3 , takes these errors into account and corrects for them in the computation of $\tilde{r}_{23}$. This happens again in the computation of q_4, q_5 , and so on. As a consequence, in the presence of roundoff errors, the modified Gram-Schmidt process produces vectors that are more nearly orthogonal than those produced by the

classical process. The following exercise provides a dramatic example, and more examples are given below.

Exercise 3.4.26 Let

$$v_1 = \begin{bmatrix} 1 \\ \epsilon \\ 0 \\ 0 \end{bmatrix} \quad v_2 = \begin{bmatrix} 1 \\ 0 \\ \epsilon \\ 0 \end{bmatrix} \quad v_3 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \epsilon \end{bmatrix},$$

where $\epsilon \ll 1$. Note that these vectors are very nearly linearly dependent. Suppose ϵ is so small that $\epsilon^2 < u$, where u is the unit roundoff error. Then $\text{fl}(1 + \epsilon^2) = 1$.

- (a) Use the classical Gram-Schmidt process to orthogonalize these vectors, making the approximation $1 + \epsilon^2 = 1$. Although a computer would make additional rounding errors, you may do the rest of the computations exactly for simplicity. Note that the computed q_2 and q_3 satisfy $\langle q_2, q_3 \rangle = 1/2$. They are far from orthogonal.
- (b) Repeat part (a) using the modified Gram-Schmidt process, and note that the computed q_2 and q_3 satisfy $\langle q_2, q_3 \rangle = 0$. Since $\langle q_1, q_2 \rangle = -\epsilon/\sqrt{2}$ and $\langle q_1, q_3 \rangle = -\epsilon/\sqrt{6}$, we see that the modified Gram-Schmidt algorithm has done a decent job of producing orthonormal vectors from these highly ill conditioned starting vectors. The whole difference is in the computation of r_{23} .

□

Efficacy of Least-Squares Solvers

Because of the good numerical properties of reflectors and rotators, the QR decomposition method using either of these types of transformations solves the least squares problem (full rank case) with as much accuracy as we could hope for. It turns out that the modified Gram-Schmidt method solves least squares problems equally accurately [7]. As Exercise 3.4.43 shows, the modified Gram-Schmidt method has a slightly higher flop count than the QR decomposition using reflectors.

Efficacy of Orthonormalizers

The Gram-Schmidt procedure was originally presented as an algorithm for orthonormalizing vectors. Only after the connection with the QR decomposition had been made did we begin to view it as a method for solving the least squares problem. We can also reverse our viewpoint and view the QR decomposition algorithm using reflectors as a method for orthonormalizing vectors. If we want to use that algorithm for this task, additional work needs to be done: The orthogonal matrix Q is provided implicitly, as a product of reflectors. If we want the orthonormal vectors, we need to assemble Q . It turns out that this additional work doubles the flop count of the

algorithm, giving it a higher flop count than the modified Gram-Schmidt method (See Exercise 3.4.44). Of course, one should never forget that the flop count gives only a rough idea of the cost of the algorithm. The main message is that the costs of running these various algorithms do not differ by much.

In the face of roundoff errors we cannot expect our algorithms to produce vectors that are exactly orthogonal. At best we can hope that $|\langle q_i, q_j \rangle| \leq cu$ for $i \neq j$, where cu is a modest multiple of the unit roundoff u . We will find it convenient to have a measure of deviation from orthonormality. The vectors $q_1, \dots, q_m$ are orthonormal if and only if the matrix $Q = [q_1 \cdots q_m]$ is isometric. This means $Q^T Q = I_m$ or, equivalently, $I_m - Q^T Q = 0$. If the vectors are not orthonormal, but deviate only slightly from orthonormality, then the matrix $I_m - Q^T Q$ will be close to zero, and conversely. We therefore take $\|I_m - Q^T Q\|_2$ as our measure of deviation from orthonormality.

The excellent numerical properties of reflectors guarantee that they do a good job at orthonormalization. The vectors produced generally satisfy

$$\|I_m - Q^T Q\|_2 \approx cu,$$

a modest multiple of the unit roundoff. We could not hope to do better. In contrast the vectors produced by the modified Gram-Schmidt algorithm deviate from orthogonality roughly in proportion with a condition number associated with the vectors $v_1, \dots, v_m$. Specifically, let $V = [v_1 \cdots v_m] \in \mathbb{R}^{n \times m}$. The condition number of a nonsquare matrix with linearly independent columns can be defined by

$$\kappa(V) = \frac{\max_{x \neq 0} \|Vx\|/\|x\|}{\min_{x \neq 0} \|Vx\|/\|x\|} = \frac{\text{maxmag}(V)}{\text{minmag}(V)}.$$

The vectors produced by the modified Gram-Schmidt process satisfy

$$\|I_m - Q^T Q\|_2 \approx u\kappa_2(V).$$

See [7] and [41]. Intuitively $\kappa(V)$ is large whenever $v_1, \dots, v_m$ are nearly linearly dependent. Thus if the modified Gram-Schmidt algorithm is applied to vectors that are nearly linearly dependent, the resulting vectors can deviate significantly from orthogonality.

Example 3.4.27 Let V be the 30×20 matrix defined by $v_{ij} = p_j^{i-1}$, $i = 1, \dots, 30$, $j = 1, \dots, 20$, where $p_j = j/20$. Matrices with entries of the form p_j^{i-1} are called *Vandermonde* matrices. According to MATLAB, $\kappa_2(V) \approx 3 \times 10^{13}$, indicating that the columns of V are nearly dependent. Using MATLAB, we orthonormalized the columns of V by three different methods and obtained the following results.

Method	$\ I_m - Q^T Q\ _2$
classical Gram-Schmidt	12.4
modified Gram-Schmidt	7.9×10^{-4}
QR decomposition by reflectors	1.9×10^{-15}

We see that the vectors produced by the classical process were nowhere near orthonormal, but the modified Gram-Schmidt process did fairly well. Since $\kappa_2(V) \approx 3 \times 10^{13}$ and $u \approx 10^{-16}$ (IEEE double-precision), we expect that $\|I_m - Q^T Q\|_2$ would be roughly 3×10^{-3} in this case. In fact it was a bit smaller. The best results were from the QR decomposition by reflectors, for which we obtained $\|I_m - Q^T Q\|_2 \approx 19u$. $\square$

Reorthogonalization

Suppose $\kappa_2(V)$ is large but not too large, say $1 \ll \kappa_2(V) \ll 1/u$. Then $u\kappa_2(V) \ll 1$, and we expect the modified Gram-Schmidt process to return vectors that satisfy $\|I_m - Q^T Q\|_2 \ll 1$, i.e. that are not too far from orthonormal. That is what happened in Example 3.4.27. If these are not yet close enough to orthonormal for our purposes, there is nothing to stop us from applying the orthonormalization process again. That is, if $\tilde{Q}$ denotes the output from our first orthonormalization, we can use $\tilde{Q}$ as the input for a second run of the modified Gram-Schmidt algorithm. Since $\tilde{Q}$ is nearly an isometry, it must be very well conditioned ($\max\text{mag}(\tilde{Q})/\min\text{mag}(\tilde{Q}) \approx 1$). Thus the output of the second run will satisfy $\|I_m - \tilde{Q}^T \tilde{Q}\|_2 \approx u\kappa_2(\tilde{Q}) \approx u$. That is, the vectors will be as close to orthonormal as we could hope for. In summary, reorthogonalization works, and generally one reorthogonalization is enough. Of course, reorthogonalization doubles the flop count.

If one is committed to reorthogonalization from the outset, then a better reorthogonalization procedure would get each vector right to begin with. At step k one transforms v_k to $v_k^{(k-1)}$, which should be orthogonal to $q_1, \dots, q_{k-1}$. In practice one can check whether it is or not by calculating the inner products $\langle v_k^{(k-1)}, q_i \rangle$ for $i = 1, \dots, k-1$. But if one is going to take the time to make these (or similar) computations, then one might as well use them to make further corrections to $v_k^{(k-1)}$. In other words, one might as well do the orthogonalization twice at each step. Our observations from the previous paragraph suggest that two passes should be enough to make q_k orthogonal to $q_1, \dots, q_{k-1}$ to full machine precision. Here is a simple but effective modified Gram-Schmidt algorithm with reorthogonalization:

Modified Gram-Schmidt with Reorthogonalization. Given linearly independent $v_1, v_2, \dots, v_m$, this algorithm produces orthonormal $q_1, q_2, \dots, q_m$ such that $\text{span}\{q_1, \dots, q_i\} = \text{span}\{v_1, \dots, v_i\}$ for $i = 1, \dots, m$. $q_1, \dots, q_m$ are stored over $v_1, \dots, v_m$.

$$\begin{array}{l}
 \text{for } k = 1, \dots, m \\
 \quad \left[\begin{array}{l}
 \text{for } i = 1, \dots, k-1 \\
 \quad \left[\begin{array}{l}
 r_{ik} \leftarrow \langle v_k, v_i \rangle \\
 v_k \leftarrow v_k - v_i r_{ik}
 \end{array} \right. \\
 \text{for } i = 1, \dots, k-1 \\
 \quad \left[\begin{array}{l}
 \alpha \leftarrow \langle v_k, v_i \rangle \\
 v_k \leftarrow v_k - v_i \alpha \\
 r_{ik} \leftarrow r_{ik} + \alpha
 \end{array} \right. \\
 r_{kk} \leftarrow \|v_k\|_2 \\
 \text{if } (r_{kk} = 0) \text{ set flag } (v_1, \dots, v_k \text{ are dependent}), \text{ exit} \\
 v_k \leftarrow (1/r_{kk})v_k
 \end{array} \right.
 \end{array} \tag{3.4.28}$$

This is the same as (3.4.25), except that the orthogonalization loop is done twice. The second time through, each coefficient α is added to the appropriate r_{ik} to maintain the relationship $V = QR$. If we apply this algorithm to the matrix in Example 3.4.27, we obtain the excellent result $\|I_m - Q^T Q\| \approx 1.0 \times 10^{-15} \approx 9u$.

Exercise 3.4.29 Use several different methods to orthogonalize the columns of the 7×7 Hilbert matrix ($V = \text{hilib}(m)$). In each case check the effectiveness of the procedure by computing $\|I_m - Q^T Q\|_2$ (`norm(eye(m) - Q' * Q)`). Check the residual $\|V - QR\|_2$ as well. This exercise is easier than it looks. Just make MATLAB m-files implementing algorithms (3.4.19), (3.4.25), and (3.4.28). They are all quite similar.

- (a) Classical Gram-Schmidt.
- (b) Modified Gram-Schmidt. Did you get what you would expect (that is, what I told you to expect), in light of the values of u and $\kappa_2(V)$?
- (c) Modified Gram-Schmidt (3.4.25) applied twice.
- (d) Modified Gram-Schmidt with reorthogonalization (3.4.28).
- (e) QR decomposition with reflectors. (For this one just use MATLAB's `qr` command: `[Q, R] = qr(V, 0)`.)

□

Exercise 3.4.30 Repeat Exercise 3.4.29 using the 12×12 Hilbert matrix.

□

Exercise 3.4.31 One possibility we have not yet considered is *classical* Gram-Schmidt with reorthogonalization. Try the following variants on both the 7×7 and 12×12 Hilbert matrices.

- (a) Classical Gram-Schmidt (3.4.19) applied repeatedly until full machine precision is reached.
- (b) Classical Gram Schmidt with reorthogonalization, in the spirit of (3.4.28). This variant works surprisingly well.

□

Additional Exercises

Exercise 3.4.32 In this exercise we discuss various ways the classical Gram-Schmidt algorithm can be implemented in MATLAB. However, the points illustrated here can be applied to other algorithms and other modern computing languages, for example, the newer versions of Fortran (Fortran 90 and later). In the classical Gram-Schmidt algorithm (3.4.19) we calculate the inner products $r_{ik} \leftarrow \langle v_k, v_i \rangle$. Suppose the vectors are packed into a matrix $V = [v_1 \cdots v_m]$. One way to accumulate the inner product in MATLAB is

```

r(ii, kk) = 0;
for jj = 1:n
    r(ii, kk) = r(ii, kk) + v(jj, kk)*v(jj, ii);
end

```

In the early days of computing, this was the usual way to code this task. Modern computing languages, including MATLAB, have higher level instructions that make the task easier. In MATLAB (and Fortran 90) the k th column of V can be written as $V(:, k)$ or $V(1:n, k)$, and the inner product can be executed as a matrix product:

```

r(ii, kk) = v(:, ii)' * v(:, kk);

```

This code is simpler, and it also runs much faster in MATLAB.⁷

- (a) In (3.4.19) the instruction that computes r_{ik} is embedded in a loop that calculates r_{ik} for $i = 1, \dots, k-1$. These entries can be accessed as a single vector as $r(1:k-1, k)$. Show how to replace the loop by a single matrix product.
- (b) The second i loop in (3.4.19) updates the vector v_k . Write this in three different ways using MATLAB: (i) using two nested loops (the “old-fashioned” way), (ii) using a single i loop to update the vector (more or less as it is written in (3.4.19)), and (iii) as a single matrix operation:

```

v(:, k) = v(:, k) + ....

```

This gives the most compact, elegant, and also fastest code.

- (c) To what extent can these ideas be applied to the modified Gram-Schmidt algorithm as formulated in (3.4.25)? A form of modified Gram-Schmidt that is more amenable to elegant coding is introduced in the following Exercise.

□

Exercise 3.4.33 We presented the modified Gram-Schmidt algorithm (3.4.25) in such a way that it resembles the classical algorithm (3.4.19) as much as possible. This organization is different from what one usually sees. Usually, as soon as q_1 is available, the coefficients $r_{12}, \dots, r_{1m}$ are computed, and the component $r_{1j}q_1$ is subtracted from v_j , $j = 1, \dots, m$. This gives vectors $v_2^{(1)}, \dots, v_m^{(1)}$, all of which are orthogonal to q_1 . Then q_2 is computed by normalizing the vector $v_2^{(1)} = v_2 - q_1 r_{12}$, the vectors $v_3^{(1)}, \dots, v_m^{(1)}$ are orthogonalized against q_2 , and so on.

- (a) Write out this algorithm in pseudocode.

⁷However, you will not notice the difference for this small bit of code unless n is made quite large.

Solution:

$$\begin{array}{l}
 \text{for } k = 1, \dots, m \\
 \left[\begin{array}{l}
 r_{kk} \leftarrow \|v_k\|_2 \\
 \text{if } (r_{kk} = 0) \text{ set flag } (v_1, \dots, v_k \text{ are dependent}), \text{ exit} \\
 v_k \leftarrow (1/r_{kk})v_k \\
 \text{for } j = k+1, \dots, m \quad (\text{skipped when } j = m) \\
 \quad \left[r_{kj} \leftarrow \langle v_j, v_k \rangle \quad (v_k \text{ is really } q_k.) \right. \\
 \text{for } j = k+1, \dots, m \\
 \quad \left. \left[v_j \leftarrow v_j - v_k r_{kj} \right] \right.
 \end{array} \right. \quad (3.4.34)
 \end{array}$$

- (b) Show that this algorithm is identical to the modified Gram-Schmidt process (3.4.25), except for the order in which the operations are performed. (In particular, the two algorithms have the same roundoff errors.)
- (c) Write a MATLAB version of (3.4.34). Express each of the j loops as a single matrix operation. (For example, the vectors $v_{k+1}, \dots, v_m$ can be accessed at once as $V(:, k+1:m)$.)

□

Exercise 3.4.35 In Chapter 1 we saw that the various ways of calculating the LU decomposition can be derived by partitioning the equation $A = LU$ in different ways. It is natural to try to do the same thing with the QR decomposition. Consider a decomposition $V = QR$, where $V = [v_1 \cdots v_k] \in \mathbb{R}^{n \times k}$, $n \geq k$, and V has full rank; $Q = [q_1 \cdots q_k] \in \mathbb{R}^{n \times k}$ is an isometry; and $R \in \mathbb{R}^{k \times k}$ is upper triangular with positive entries on the main diagonal. Partition the matrices as

$$V = [\tilde{V} \mid v_k], \quad Q = [\tilde{Q} \mid q_k], \quad \text{and} \quad R = \left[\begin{array}{c|c} \tilde{R} & s \\ \hline 0 & r_{kk} \end{array} \right].$$

Derive an algorithm to calculate q_k , given $q_1, \dots, q_{k-1}$. You may take the following steps:

- (a) Use the equation $V = QR$ in partitioned form to derive a formula for $q_k r_{kk}$ in terms of known quantities and s .
- (b) The condition that q_k is orthogonal to $q_1, \dots, q_{k-1}$ can be written as $\tilde{Q}^T q_k = 0$. Use this equation to derive a formula for s .
- (c) Show that q_k and r_{kk} are uniquely determined by $q_k r_{kk}$ and the conditions $\|q_k\|_2 = 1$ and $r_{kk} > 0$.
- (d) Parts (a), (b), and (c) can be combined to yield an algorithm to calculate q_k , given $q_1, \dots, q_{k-1}$. Show that this is exactly the classical Gram-Schmidt algorithm.

□

Exercise 3.4.36 Let V , Q , and R be as in the previous exercise, and consider the partition

$$V = \begin{bmatrix} v_1 & | & \tilde{V} \end{bmatrix}, \quad Q = \begin{bmatrix} q_1 & | & \tilde{Q} \end{bmatrix}, \quad \text{and} \quad R = \left[\begin{array}{c|c} r_{11} & r^T \\ \hline 0 & \tilde{R} \end{array} \right].$$

- Use this partition to derive an algorithm for calculating Q and R .
- Show that this algorithm is exactly the modified Gram-Schmidt process in the form (3.4.34).

One can also derive a block modified Gram-Schmidt algorithm by this method. $\square$

Exercise 3.4.37 State and prove a complex version of Theorem 3.4.2. $\square$

Exercise 3.4.38 In this exercise you will show that the subspace equalities

$$\text{span}\{q_1, \dots, q_k\} = \text{span}\{v_1, \dots, v_k\}$$

hold in the Gram-Schmidt process.

- Use (3.4.15) and (3.4.18) to show that $q_k \in \text{span}\{q_1, \dots, q_{k-1}, v_k\}$.
- Show that if $\text{span}\{q_1, \dots, q_{k-1}\} = \text{span}\{v_1, \dots, v_{k-1}\}$, then

$$\text{span}\{q_1, \dots, q_{k-1}, v_k\} = \text{span}\{v_1, \dots, v_{k-1}, v_k\}.$$

- Verify (3.4.20), and conclude that $v_k \in \text{span}\{q_1, \dots, q_k\}$.
- Using the observations from parts (a), (b), and (c), prove by induction on k that $\text{span}\{q_1, \dots, q_k\} = \text{span}\{v_1, \dots, v_k\}$.

$\square$

Exercise 3.4.39 Another way to demonstrate the subspace equalities is to make use of the QR decomposition. Let $v_1, \dots, v_m$ and $q_1, \dots, q_m$ be two linearly independent sets of vectors, let $V = [v_1 \dots v_m] \in \mathbb{R}^{n \times m}$, and let $Q = [q_1 \dots q_m] \in \mathbb{R}^{n \times m}$.

- Show that if $V = QR$, where $R \in \mathbb{R}^{m \times m}$ is upper triangular, then

$$\text{span}\{v_1, \dots, v_k\} \subseteq \text{span}\{q_1, \dots, q_k\} \quad \text{for } k = 1, \dots, m.$$

- (Review) Show that if R is upper triangular and nonsingular, then R^{-1} is also upper triangular.
- Show that if $V = QR$, where R is upper triangular and nonsingular, then

$$\text{span}\{q_1, \dots, q_k\} \subseteq \text{span}\{v_1, \dots, v_k\} \quad \text{for } k = 1, \dots, m.$$

- From (a) and (c), deduce that the vectors produced by the Gram-Schmidt process satisfy

$$\text{span}\{q_1, \dots, q_k\} = \text{span}\{v_1, \dots, v_k\} \quad \text{for } k = 1, \dots, m.$$

(e) Conversely, show that if

$$\text{span}\{q_1, \dots, q_k\} = \text{span}\{v_1, \dots, v_k\} \quad \text{for } k = 1, \dots, m,$$

then there is a nonsingular, upper triangular matrix $R \in \mathbb{R}^{m \times m}$ such that $V = QR$.

□

Exercise 3.4.40 (Modified Gram-Schmidt = Classical Gram Schmidt)

(a) Referring to the modified Gram-Schmidt step (3.4.23), show that for $k = 1, \dots, m$,

$$v_k^{(i-1)} = v_k - \sum_{j=1}^{i-1} \tilde{r}_{ik} \tilde{q}_j \quad i = 1, \dots, k.$$

(b) The modified Gram-Schmidt algorithm breaks down at step k if $\tilde{r}_{kk} = 0$. Assuming that $v_1, \dots, v_m$ are linearly independent, prove by induction on k that $\tilde{r}_{kk} \neq 0$ and $\text{span}\{\tilde{q}_1, \dots, \tilde{q}_k\} = \text{span}\{v_1, \dots, v_k\}$ for $k = 1, \dots, m$.

(c) Let $V = [v_1 \cdots v_m] \in \mathbb{R}^{n \times m}$, $\tilde{Q} = [\tilde{q}_1 \cdots \tilde{q}_m] \in \mathbb{R}^{n \times m}$, and

$$\tilde{R} = \begin{bmatrix} \tilde{r}_{11} & \cdots & \tilde{r}_{1m} \\ & \ddots & \vdots \\ 0 & & \tilde{r}_{mm} \end{bmatrix} \in \mathbb{R}^{m \times m}.$$

Then $\tilde{Q}$ is isometric, and $\tilde{R}$ is upper triangular with positive entries on the main diagonal. Show that $V = \tilde{Q}\tilde{R}$. Conclude that the vectors $\tilde{q}_1, \dots, \tilde{q}_m$ and coefficients $\tilde{r}_{ij}$ produced by the modified Gram-Schmidt process are identical to those produced by the classical Gram-Schmidt process.

□

Exercise 3.4.41 In Exercise 3.4.40 you invoked the uniqueness of the QR decomposition to prove that the two forms of the Gram-Schmidt algorithm are equivalent in principle. This is a convenient approach. Of course, one can also prove the result directly. Prove by induction on k that for $k = 1, \dots, m$,

$$\begin{aligned} \tilde{r}_{ik} &= r_{ik} & i = 1, \dots, k-1 \\ v_k^{(k-1)} &= \tilde{q}_k \\ \tilde{r}_{kk} &= r_{kk} \\ \tilde{q}_k &= q_k. \end{aligned}$$

□

Exercise 3.4.42 When one computes a QR decomposition in practice, one does not usually bother to force the main-diagonal elements of R to be positive. In this case the

columns of Q may differ from the Gram-Schmidt vectors, but it is not hard to show that the difference is trivial: Let $v_1, \dots, v_m$ be linearly independent vectors, and let $V = [v_1 \dots v_m]$. Suppose $V = QR$, where $Q \in \mathbb{R}^{n \times m}$ is an isometry and $R \in \mathbb{R}^{m \times m}$ is upper triangular but does not necessarily have positive main-diagonal entries. Let $q_1, \dots, q_m$ denote the columns of Q , and let $\hat{q}_1, \dots, \hat{q}_m$ denote the vectors obtained from $v_1, \dots, v_m$ by the Gram-Schmidt process. Show that $q_i = \pm \hat{q}_i$ for $i = 1, \dots, m$. $\square$

Exercise 3.4.43 Show that the modified Gram-Schmidt algorithm takes about $2nm^2$ flops to orthonormalize m vectors in $\mathbb{R}^n$. Compare this figure with the cost of performing a QR decomposition using reflectors in two cases: (i) $n \gg m$ and (ii) $n = m$. $\square$

Exercise 3.4.44 If we want to use the QR decomposition by reflectors as a method to orthonormalize vectors, we need to assemble the matrix Q , which is a product of m reflectors: $Q = Q_1 Q_2 \dots Q_m$.

- Clearly $q_1 = Qe_1 = Q_1 Q_2 \dots Q_m e_1$. Taking into consideration the form of the reflectors $Q_2, \dots, Q_m$, show that $q_1 = Q_1 e_1$. Thus it costs $4n$ flops to compute q_1 .
- Generalizing part (a), show that $q_i = Q_1 \dots Q_i e_i$ for $i = 1, \dots, m$. How many flops does this require?
- Show that the operation of computing $q_1, \dots, q_m$ costs $2nm^2 - \frac{2}{3}m^2$ flops in all, which is as much as the QR decomposition cost to begin with.

$\square$

Exercise 3.4.45 Let $A \in \mathbb{R}^{n \times m}$, $n \geq m$, and suppose $\text{rank}(A) = r$. Let $\hat{A}$ be any matrix obtained from A by permuting the columns in such a way that the first r columns are linearly independent. Prove that there exist unique $\hat{Q} \in \mathbb{R}^{n \times r}$ and $R \in \mathbb{R}^{r \times m}$ such that $\hat{Q}$ is an isometry, $R = [\hat{R} \tilde{R}]$, $\hat{R} \in \mathbb{R}^{r \times r}$ is upper triangular with positive entries on the main diagonal, and $\hat{A} = \hat{Q}R$. $\square$

Exercise 3.4.46 Let $v_1, \dots, v_m \in \mathbb{R}^n$ and $w_1, \dots, w_m \in \mathbb{R}^n$ be two linearly independent sets of vectors, and let $V = [v_1 \dots v_m] \in \mathbb{R}^{n \times m}$ and $W = [w_1 \dots w_m] \in \mathbb{R}^{n \times m}$. Show that $\text{span}\{v_1, \dots, v_i\} = \text{span}\{w_1, \dots, w_i\}$ for $i = 1, \dots, m$ if and only if there exists a nonsingular, upper triangular matrix $R \in \mathbb{R}^{m \times m}$ such that $V = WR$. $\square$

3.5 GEOMETRIC APPROACH TO THE LEAST SQUARES PROBLEM

In this section we introduce a few basic concepts and prove some fundamental theorems that yield a clear geometric picture of the least squares problem. The tools developed here will also be used later on in the book. It is traditional and possibly more logical to place this material at the beginning of the chapter, but that arrangement would have caused an unnecessary delay in the introduction of the algorithms.

Let $\mathcal{S}$ be any subset of $\mathbb{R}^n$. The *orthogonal complement* of $\mathcal{S}$, denoted $\mathcal{S}^\perp$ (pronounced $\mathcal{S}$ perp), is defined to be the set of vectors in $\mathbb{R}^n$ that are orthogonal to $\mathcal{S}$. That is,

$$\mathcal{S}^\perp = \{x \in \mathbb{R}^n \mid \langle x, y \rangle = 0 \text{ for all } y \in \mathcal{S}\}.$$

The set $\mathcal{S}^\perp$ is nonempty since it contains at least the vector 0.

Exercise 3.5.1 Show that $\mathcal{S}^\perp$ is a subspace of $\mathbb{R}^n$; that is,

- (a) show that the sum of two members of $\mathcal{S}^\perp$ is also in $\mathcal{S}^\perp$, and
- (b) show that every scalar multiple of a member of $\mathcal{S}^\perp$ is also a member of $\mathcal{S}^\perp$.

□

Exercise 3.5.2 Let $q_1, \dots, q_n$ be an orthonormal basis of $\mathbb{R}^n$, and let

$$\mathcal{S} = \text{span}\{q_1, \dots, q_k\},$$

where $1 \leq k \leq n-1$. Show that $\mathcal{S}^\perp = \text{span}\{q_{k+1}, \dots, q_n\}$.

□

Theorem 3.5.3 Let $\mathcal{S}$ be any subspace of $\mathbb{R}^n$. Then for every $x \in \mathbb{R}^n$, there exist unique elements $s \in \mathcal{S}$ and $s^\perp \in \mathcal{S}^\perp$ for which $x = s + s^\perp$.

Proof. Let $v_1, \dots, v_k$ be a basis for $\mathcal{S}$. From elementary linear algebra we know that there exist vectors $v_{k+1}, \dots, v_n$ such that $v_1, \dots, v_k, v_{k+1}, \dots, v_n$ is a basis of $\mathbb{R}^n$. Let $q_1, \dots, q_n$ be the orthonormal basis of $\mathbb{R}^n$ obtained by applying the Gram-Schmidt procedure to $v_1, \dots, v_n$. Then the nested subspace property (3.4.12) of the Gram-Schmidt process guarantees that $\text{span}\{q_1, \dots, q_k\} = \text{span}\{v_1, \dots, v_k\}$. Thus, by Exercise 3.5.2,

$$\mathcal{S} = \text{span}\{q_1, \dots, q_k\} \quad \text{and} \quad \mathcal{S}^\perp = \text{span}\{q_{k+1}, \dots, q_n\}.$$

Let $x \in \mathbb{R}^n$. We must find $s \in \mathcal{S}$ and $s^\perp \in \mathcal{S}^\perp$ such that $x = s + s^\perp$. With the help of the basis $q_1, \dots, q_n$, this is easy. x can be expressed uniquely as a linear combination of these vectors: $x = c_1 q_1 + \dots + c_n q_n$. Let $s = c_1 q_1 + \dots + c_k q_k$ and $s^\perp = c_{k+1} q_{k+1} + \dots + c_n q_n$. Then $s \in \mathcal{S}$, $s^\perp \in \mathcal{S}^\perp$, and $x = s + s^\perp$.

To see that this decomposition is unique, suppose we have another decomposition $x = \hat{s} + \hat{s}^\perp$, where $\hat{s} \in \mathcal{S}$ and $\hat{s}^\perp \in \mathcal{S}^\perp$. We will show that $\hat{s} = s$ and $\hat{s}^\perp = s^\perp$. Let $r = s - \hat{s} \in \mathcal{S}$. Then the equation $s + s^\perp = \hat{s} + \hat{s}^\perp$ implies that $r = \hat{s}^\perp - s^\perp \in \mathcal{S}^\perp$. Thus $r \in \mathcal{S} \cap \mathcal{S}^\perp$. As you can easily verify, $\mathcal{S} \cap \mathcal{S}^\perp = \{0\}$. Therefore $r = 0$, which implies $\hat{s} = s$ and $\hat{s}^\perp = s^\perp$. □

The unique elements s and $s^\perp$ whose existence is guaranteed by Theorem 3.5.3 are called the *orthogonal projections* of x into $\mathcal{S}$ and $\mathcal{S}^\perp$, respectively.

Given two subspaces $\mathcal{S}_1$ and $\mathcal{S}_2$, the *sum* $\mathcal{S}_1 + \mathcal{S}_2$ is defined by

$$\mathcal{S}_1 + \mathcal{S}_2 = \{s_1 + s_2 \mid s_1 \in \mathcal{S}_1, s_2 \in \mathcal{S}_2\}.$$

You can easily verify that $S_1 + S_2$ is a subspace of $\mathbb{R}^n$. If $U = S_1 + S_2$, then every $u \in U$ can be expressed as a sum $u = s_1 + s_2$, where $s_1 \in S_1$ and $s_2 \in S_2$. The sum $U = S_1 + S_2$ is said to be a *direct sum* if for every $u \in U$ the decomposition $u = s_1 + s_2$ is unique. Direct sums are denoted $S_1 \oplus S_2$. Theorem 3.5.3 states that for any subspace S , $\mathbb{R}^n$ is the direct sum of S and $S^\perp$:

$$\mathbb{R}^n = S \oplus S^\perp.$$

Exercise 3.5.4 Let S_1 and S_2 be subspaces of $\mathbb{R}^n$.

- (a) Prove that $S_1 + S_2$ is a subspace of $\mathbb{R}^n$.
- (b) Prove that $S_1 + S_2$ is a direct sum if and only if $S_1 \cap S_2 = \{0\}$.

□

Exercise 3.5.5 Let $v_1 = [1, 0, 0]^T$, $v_2 = [0, 1, 1]^T$, and $S = \text{span}\{v_1, v_2\} \subseteq \mathbb{R}^3$.

- (a) Find (a basis for) $S^\perp$.
- (b) Find (a basis for) a subspace U of $\mathbb{R}^3$ such that $U \neq S^\perp$ but $S \oplus U = \mathbb{R}^3$.

□

Exercise 3.5.6 Let S be a subspace of $\mathbb{R}^n$, and let $U = S^\perp$. Show that $U^\perp = S$. In other words, $S^{\perp\perp} = S$.

□

Let $A \in \mathbb{R}^{n \times m}$. Then A can be viewed as a linear transformation mapping $\mathbb{R}^m$ into $\mathbb{R}^n$: The vector $x \in \mathbb{R}^m$ is mapped to $Ax \in \mathbb{R}^n$. Two fundamental subspaces associated with a linear transformation are its null space (also known as the kernel) and its range. The *null space* of A , denoted $\mathcal{N}(A)$, is a subspace of $\mathbb{R}^m$ defined by

$$\mathcal{N}(A) = \{x \in \mathbb{R}^m \mid Ax = 0\}.$$

The *range* of A , denoted $\mathcal{R}(A)$, is a subspace of $\mathbb{R}^n$ defined by

$$\mathcal{R}(A) = \{Ax \mid x \in \mathbb{R}^m\}.$$

Exercise 3.5.7

- (a) Show that $\mathcal{N}(A)$ is a subspace of $\mathbb{R}^m$.
- (b) Show that $\mathcal{R}(A)$ is a subspace of $\mathbb{R}^n$.

□

Exercise 3.5.8 Show that the rank of A is the dimension of $\mathcal{R}(A)$.

□

Lemma 3.5.9 Let $A \in \mathbb{R}^{n \times m}$. Then for all $x \in \mathbb{R}^m$ and $y \in \mathbb{R}^n$

$$\langle Ax, y \rangle = \langle x, A^T y \rangle.$$

Proof. $\langle Ax, y \rangle = y^T(Ax) = (y^T A)x = (A^T y)^T x = \langle x, A^T y \rangle$. $\square$

Note that the inner product on the left is the inner product on $\mathbb{R}^n$, while that on the right is the inner product in $\mathbb{R}^m$.

The matrix $A^T \in \mathbb{R}^{m \times n}$ can be viewed as a linear transformation mapping $\mathbb{R}^n$ into $\mathbb{R}^m$. Thus it has a null space $\mathcal{N}(A^T) \in \mathbb{R}^n$ and a range $\mathcal{R}(A^T) \in \mathbb{R}^m$. There is an important relationship between these spaces and the corresponding spaces associated with A .

Theorem 3.5.10 $\mathcal{R}(A)^\perp = \mathcal{N}(A^T)$.

Proof. If $y \in \mathcal{R}(A)^\perp$, then $\langle Ax, y \rangle = 0$ for all $x \in \mathbb{R}^m$, so by Lemma 3.5.9, $\langle x, A^T y \rangle = 0$ for all $x \in \mathbb{R}^m$. Thus in particular we can take $x = A^T y$ and get $\langle A^T y, A^T y \rangle = 0$, which implies $\|A^T y\|_2^2 = 0$, $A^T y = 0$, and $y \in \mathcal{N}(A^T)$. Therefore $\mathcal{R}(A)^\perp \subseteq \mathcal{N}(A^T)$. The reverse inclusion $\mathcal{N}(A^T) \subseteq \mathcal{R}(A)^\perp$ is left as an exercise for you. $\square$

Exercise 3.5.11 Prove that $\mathcal{N}(A^T) \subseteq \mathcal{R}(A)^\perp$. $\square$

Applying the result of Exercise 3.5.6, we see that an alternative statement of Theorem 3.5.10 is $\mathcal{N}(A^T)^\perp = \mathcal{R}(A)$. Since $(A^T)^T = A$, the roles of A and A^T can be reversed in Theorem 3.5.10 to yield the following corollary.

Corollary 3.5.12 $\mathcal{N}(A)^\perp = \mathcal{R}(A^T)$.

An equivalent statement is $\mathcal{R}(A^T)^\perp = \mathcal{N}(A)$. Finally

$$\mathbb{R}^n = \mathcal{R}(A) \oplus \mathcal{N}(A^T),$$

$$\mathbb{R}^m = \mathcal{N}(A) \oplus \mathcal{R}(A^T).$$

In the important special case in which $A \in \mathbb{R}^{n \times n}$ is symmetric, we have $\mathcal{N}(A)^\perp = \mathcal{R}(A)$ and $\mathbb{R}^n = \mathcal{N}(A) \oplus \mathcal{R}(A)$.

Exercise 3.5.13

- The *column space* of A is defined to be the subspace of $\mathbb{R}^n$ spanned by the columns of A . Obviously the dimension of the column space is just the rank of A . Prove that the column space of A is $\mathcal{R}(A)$.
- The *row space* of A is defined to be the subspace of $\mathbb{R}^m$ spanned by the rows of A . Prove that the row space of A is just $\mathcal{N}(A)^\perp$. Use the equation $Ax = 0$; do not use Theorem 3.5.10 or Corollary 3.5.12.
- Obviously the column space of A is the same as the row space of A^T . Use this observation, along with the results of parts (a) and (b), to obtain a second proof of Theorem 3.5.10 and Corollary 3.5.12.

$\square$

The Discrete Least Squares Problem

Let $A \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^n$, $n \geq m$, and consider the least squares problem for the overdetermined system $Ax = b$.⁸ The problem is to find $x \in \mathbb{R}^m$ such that

$$\|b - Ax\|_2 = \min_{w \in \mathbb{R}^m} \|b - Aw\|_2.$$

The set of all Aw such that $w \in \mathbb{R}^m$ is $\mathcal{R}(A)$, so this problem is obviously closely related to that of finding $y \in \mathcal{R}(A)$ such that

$$\|b - y\|_2 = \min_{s \in \mathcal{R}(A)} \|b - s\|_2.$$

The next theorem shows that this problem has a unique solution, and it gives a simple characterization of the solution. First we have to prove a lemma.

Lemma 3.5.14 (*Pythagorean Theorem*) *Let u and v be orthogonal vectors in $\mathbb{R}^n$. Then*

$$\|u + v\|_2^2 = \|u\|_2^2 + \|v\|_2^2.$$

Proof. $\|u + v\|_2^2 = \langle u + v, u + v \rangle = \langle u, u \rangle + \langle v, u \rangle + \langle u, v \rangle + \langle v, v \rangle = \langle u, u \rangle + \langle v, v \rangle$, because $\langle u, v \rangle = \langle v, u \rangle = 0$. Thus $\|u + v\|_2^2 = \|u\|_2^2 + \|v\|_2^2$. $\square$

Theorem 3.5.15 *Let S be a subspace of $\mathbb{R}^n$, and let $b \in \mathbb{R}^n$. Then there exists a unique $y \in S$ such that*

$$\|b - y\|_2 = \min_{s \in S} \|b - s\|_2. \quad (3.5.16)$$

y is the unique element in S such that $b - y \in S^\perp$. In other words, y is the orthogonal projection of b into S .

Exercise 3.5.17 We all learned in school that the shortest distance from a point to a straight line is along the perpendicular. Theorem 3.5.15 generalizes this simple fact. Draw a picture that illustrates Theorem 3.5.15 in the case when the dimension of S is 1. Then the vectors in S all lie in a line. The shortest distance from a point (the tip of b) to a straight line (S) is along the perpendicular ($b - y$). $\square$

Proof of Theorem 3.5.15. By Theorem 3.5.3 there exist unique elements $y \in S$ and $z \in S^\perp$ such that $b = y + z$. y is the orthogonal projection of b into S . Notice that $b - y = z \in S^\perp$. There can be no other $w \in S$ such that $b - w \in S^\perp$, because then the decomposition $b = w + (b - w)$ would violate the uniqueness part of Theorem 3.5.3. To see that y satisfies (3.5.16), let $s \in S$, and consider $\|b - s\|_2^2$. Since $b - s = (b - y) + (y - s)$, where $b - y \in S^\perp$ and $y - s \in S$, we have by Lemma 3.5.14

$$\|b - s\|_2^2 = \|b - y\|_2^2 + \|y - s\|_2^2.$$

⁸In least squares problems we typically have $n \gg m$. However, the results that follow are also valid for $n < m$.

As s runs through $\mathcal{S}$, the term $\|b - y\|_2^2$ remains constant, while the term $\|y - s\|_2^2$ varies but remains strictly positive, except that it equals zero when $y = s$. Thus $\|b - s\|_2^2$, and hence also $\|b - s\|_2$, is minimized when and only when $s = y$. $\square$

Now let us see what Theorem 3.5.15 tells us about the least squares problem. Taking $\mathcal{S} = \mathcal{R}(A)$, we find that there is a unique $y \in \mathcal{R}(A)$ such that $\|b - y\|_2 = \min_{s \in \mathcal{R}(A)} \|b - s\|_2 = \min_{w \in \mathbb{R}^m} \|b - Aw\|_2$. Any $x \in \mathbb{R}^m$ that satisfies $Ax = y$ will be a solution to the least squares problem for the system $Ax = b$. Since $y \in \mathcal{R}(A)$, there must be at least one such x . This proves that the least squares problem always has at least one solution—a fact that we already proved in Section 3.3 by other means.

Exercise 3.5.18

- Suppose $x \in \mathbb{R}^m$ satisfies $Ax = y$. Show that $A\hat{x} = y$ if and only if $x - \hat{x} \in \mathcal{N}(A)$.
- Show that the least squares problem has a unique solution if and only if $\mathcal{N}(A) = \{0\}$.

$\square$

We now have two necessary and sufficient conditions for uniqueness of a solution to the least squares problem. In Section 3.3 we saw that the solution is unique if and only if A has full rank, and now we see that it is unique if and only if $\mathcal{N}(A) = \{0\}$. These two conditions must therefore be equivalent. In the following exercise you are asked to prove directly that they are.

Exercise 3.5.19 Let $A \in \mathbb{R}^{n \times m}$. Prove that $\text{rank}(A) = m$ if and only if $\mathcal{N}(A) = \{0\}$. $\square$

The part of Theorem 3.5.15 that characterizes the minimizing vector yields the following corollary.

Corollary 3.5.20 Let $x \in \mathbb{R}^m$. Then $\|b - Ax\|_2 = \min_{w \in \mathbb{R}^m} \|b - Aw\|_2$ if and only if $b - Ax \in \mathcal{R}(A)^\perp$.

Combining this corollary with Theorem 3.5.10, we see that x solves the least squares problem if and only if $b - Ax \in \mathcal{N}(A^T)$; that is, $A^T(Ax - b) = 0$. Rewriting this last equation, we obtain the following important result.

Theorem 3.5.21 Let $x \in \mathbb{R}^m$. Then x solves the least squares problem for the system $Ax = b$ if and only if

$$A^T Ax = A^T b. \quad (3.5.22)$$

The matrix $A^T A$ is in $\mathbb{R}^{m \times m}$, and the vector $A^T b$ is in $\mathbb{R}^m$, so (3.5.22) is a system of m linear equations in m unknowns. Since these are the equations that hold when $b - Ax$ is normal (meaning orthogonal) to $\mathcal{R}(A)$, they are known as the *normal equations*.

The coefficient matrix of the normal equations is *positive semidefinite*; that is, it is symmetric, and $x^T(A^T A)x \geq 0$ for all $x \in \mathbb{R}^m$. If $\text{rank}(A) = m$, then $A^T A$ is positive definite.

Exercise 3.5.23 Work this problem by hand. Consider the overdetermined system

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} x \end{bmatrix} = \begin{bmatrix} 9 \\ 5 \end{bmatrix},$$

which we considered previously in Exercise 3.3.7. Solve the least-squares problem again, but this time do it by setting up and solving the normal equations. $\square$

Exercise 3.5.24

- (a) Prove that $A^T A$ is positive semidefinite.
- (b) Suppose $A \in \mathbb{R}^{n \times m}$ with $n < m$. Show that $A^T A$ is not positive definite.
- (c) Suppose $A \in \mathbb{R}^{n \times m}$ with $n \geq m$. Show that $A^T A$ is positive definite if and only if A has full rank. (We already covered the case $n = m$ in Theorem 1.4.4.)

$\square$

In the full-rank case (with $n \geq m$) the unique solution of the least squares problem can be found by constructing the normal equations (3.5.22) and solving this positive definite system by Cholesky's method. Indeed, up until about 1970 this was the standard technique for solving least squares problems. Its advantages are that it is simple and inexpensive. The system that has to be solved is $m \times m$, and m is usually small. The biggest expense is that of computing the coefficient matrix $A^T A$.

The disadvantage of the normal equations approach is that it is sometimes less accurate than the QR approach. Critical information can be lost when $A^T A$ is formed.

Example 3.5.25 Let

$$A = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \\ 0 & \epsilon \end{bmatrix},$$

where $\epsilon > 0$ is small. Clearly A has full rank, and

$$A^T A = \begin{bmatrix} 1 + \epsilon^2 & 1 \\ 1 & 1 + \epsilon^2 \end{bmatrix},$$

which is positive definite. However, if ϵ is small enough that ϵ^2 is less than the unit roundoff u , then the computed $A^T A$ will be $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, which is singular. $\square$

In spite of its inferior numerical properties, the normal equation approach is still sometimes used to solve least squares problems. It can be used safely whenever A is

well conditioned. This issue will be covered in more detail in Section 4.4, in which we discuss the sensitivity of the least squares problem.

Exercise 3.5.26

- Let $B \in \mathbb{R}^{n \times m}$ be any matrix such that $\mathcal{R}(B) = \mathcal{R}(A)$. Show that x is a solution of the least squares problem for the overdetermined system $Ax = b$ if and only if $B^T Ax = B^T b$.
- Show that $\mathcal{R}(A) = \mathcal{R}(B)$ if and only if there exists a nonsingular $C \in \mathbb{R}^{m \times m}$ such that $A = BC$.

□

The most obvious instance of the situation in Exercise 3.5.26 is the case $B = A$, for which the system $B^T Ax = B^T b$ is just the system of normal equations (3.5.22). Another instance stems from the QR decomposition $A = Q\hat{R}$, where $Q \in \mathbb{R}^{n \times m}$ is an isometry and $\hat{R} \in \mathbb{R}^{m \times m}$ is upper triangular (Theorem 3.4.7). If A has full rank, then $\hat{R}$ is nonsingular, and by part (b) of Exercise 3.5.26, $\mathcal{R}(A) = \mathcal{R}(Q)$. Therefore, by part (a), the unique solution of the least squares problem satisfies

$$Q^T Ax = Q^T b. \quad (3.5.27)$$

Exercise 3.5.28

- Show that the system (3.5.27) is exactly the system that is solved, in the guise $\hat{R}x = \hat{c}$, when the least squares problem is solved by the QR decomposition method.
- Suppose we solve the normal equations $A^T Ax = A^T b$ using a Cholesky decomposition $A^T A = R^T R$. In this approach the last step is to solve an upper triangular system $Rx = y$ by back substitution. Prove that this system is exactly the same as the system $\hat{R}x = \hat{c}$ that arises in the QR decomposition method. (However, the two methods arrive at this system by radically different approaches.)

□

Exercise 3.5.29 (Derivation of the normal equations using calculus). The function

$$f(x_1, \dots, x_m) = f(x) = \|b - Ax\|_2^2$$

is a differentiable function of m variables. It has a minimum only when $\nabla f = (\partial f / \partial x_1, \dots, \partial f / \partial x_m)^T = 0$. Calculate ∇f , and note that the equations $\nabla f = 0$ are just the normal equations. □

The Continuous Least Squares Problem

We introduced the discrete least squares problem by considering the problem of approximating a discrete point set $\{(t_i, y_i) \mid i = 1, \dots, n\}$ by a simple curve such as a straight line. In the continuous least squares problem the discrete point set is replaced by continuous data $\{(t, f(t)) \mid t \in [a, b]\}$. Thus, given a function f defined on some interval $[a, b]$, we seek a simple function ϕ (e.g. a linear polynomial) such that ϕ approximates f well on $[a, b]$. The goodness of approximation is measured, not by calculating a *sum* of squares, but by calculating the *integral* of the square of the error:

$$\int_a^b |f(x) - \phi(x)|^2 dx. \quad (3.5.30)$$

The continuous least squares problem is solved by minimizing this integral over whichever set of functions ϕ we are allowing as approximations of f . For example, if the approximating function is to be a first-degree polynomial, we minimize (3.5.30) over the set of functions $\{\phi \mid \phi(t) = a_0 + a_1 t; a_0, a_1 \in \mathbb{R}\}$. More generally, if the approximating function is to be a polynomial of degree less than m , we minimize (3.5.30) over the set of functions

$$\mathcal{P}_{m-1} = \left\{ \phi \mid \phi(t) = \sum_{k=0}^{m-1} a_k t^k; a_k \in \mathbb{R}, k = 0, \dots, m-1 \right\}.$$

The set $\mathcal{P}_{m-1}$ is an m -dimensional vector space of functions. Still more generally we can let $\mathcal{S}$ be any m -dimensional vector space of functions defined on $[a, b]$ and minimize (3.5.30) over $\mathcal{S}$.

We can solve the continuous least squares problem by introducing an inner product and a norm for functions and utilizing the geometric ideas introduced in this section. The *inner product* of two functions f and g on $[a, b]$ is defined by

$$\langle f, g \rangle = \int_a^b f(x)g(x) dx.$$

This inner product enjoys the same algebraic properties as the inner product on $\mathbb{R}^n$. For example, $\langle f, g \rangle = \langle g, f \rangle$, and $\langle f_1 + f_2, g \rangle = \langle f_1, g \rangle + \langle f_2, g \rangle$. Two functions f and g are said to be *orthogonal* if $\langle f, g \rangle = 0$. The *norm* of f is defined by

$$\|f\| = \left(\int_a^b |f(x)|^2 dx \right)^{1/2}.$$

Notice that the norm and inner product are related by $\|f\| = \sqrt{\langle f, f \rangle}$. Furthermore (3.5.30) can be expressed in terms of the norm as $\|f - \phi\|^2$. Thus the continuous least squares problem is to find $\phi \in \mathcal{S}$ such that

$$\|f - \phi\| = \min_{\psi \in \mathcal{S}} \|f - \psi\|. \quad (3.5.31)$$

We proved Theorem 3.5.15 only for subspaces of $\mathbb{R}^n$, but it is valid for function spaces as well. See, for example, [70, Theorem II.7.2]. Thus the continuous least squares

problem (3.5.31) has a unique solution, which is characterized by $f - \phi \in \mathcal{S}^\perp$; that is,

$$\langle f - \phi, \psi \rangle = 0 \quad \text{for all } \psi \in \mathcal{S}. \quad (3.5.32)$$

Let $\phi_1, \dots, \phi_m$ be a basis for $\mathcal{S}$. Then $\phi = \sum_{j=1}^m x_j \phi_j$ for some unknown coefficients $x_1, \dots, x_m$. Substituting this expression for ϕ into (3.5.32), setting $\psi = \phi_i$, and applying some of the basic properties of inner products, we obtain

$$\sum_{j=1}^m \langle \phi_j, \phi_i \rangle x_j = \langle f, \phi_i \rangle \quad i = 1, \dots, m. \quad (3.5.33)$$

This is a system of m linear equations in m unknowns, which can be written as a matrix equation

$$Cx = d, \quad (3.5.34)$$

where

$$C = \begin{bmatrix} \langle \phi_1, \phi_1 \rangle & \cdots & \langle \phi_m, \phi_1 \rangle \\ \vdots & & \vdots \\ \langle \phi_1, \phi_m \rangle & \cdots & \langle \phi_m, \phi_m \rangle \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \quad \text{and} \quad d = \begin{bmatrix} \langle f, \phi_1 \rangle \\ \vdots \\ \langle f, \phi_m \rangle \end{bmatrix}.$$

The matrix C is clearly symmetric. In fact, it is positive definite, so (3.5.34) can be solved by Cholesky's method to yield the solution of the continuous least squares problem.

Exercise 3.5.35 With each nonzero $y = [y_1 \cdots y_m]^T \in \mathbb{R}^m$ associate the nonzero function $\psi = \sum_{j=1}^m y_j \phi_j$. Prove that $y^T C y = \langle \psi, \psi \rangle$. Combining this with the fact that $\langle \psi, \psi \rangle > 0$, we conclude that C is positive definite. $\square$

The next exercise shows that the equations (3.5.34) are analogous to the normal equations of the discrete least squares problem.

Exercise 3.5.36 Find $v_1, \dots, v_m \in \mathbb{R}^m$ for which the normal equations (3.5.22) have the form

$$\sum_{j=1}^m \langle v_j, v_i \rangle x_j = \langle f, v_i \rangle \quad i = 1, \dots, m.$$

Thus the normal equations have the same general form as (3.5.33). $\square$

Exercise 3.5.37 Find the polynomial $\phi(t) = x_1 + x_2 t$ of degree 1 that best approximates $f(t) = t^2$ in the least squares sense on $[0, 1]$. Check your answer by verifying that $f - \phi$ is orthogonal to $\mathcal{S} = \{a_0 + a_1 t \mid a_0, a_1 \in \mathbb{R}\}$. $\square$

Exercise 3.5.38 Let $[a, b] = [0, 1]$, $\mathcal{S} = \mathcal{P}_{m-1}$, and let $\phi_1, \dots, \phi_m$ be the basis of $\mathcal{P}_{m-1}$ defined by $\phi_1(t) = 1$, $\phi_2(t) = t$, $\phi_3(t) = t^2$, $\dots$, $\phi_m(t) = t^{m-1}$. Calculate the matrix C that would appear in (3.5.34) in this case. Note that C is just the $m \times m$ member of the family of Hilbert matrices introduced in Section 2.2. This is the context in which the Hilbert matrices first arose. $\square$

In Section 2.2 the Hilbert matrices served as an example of a family of ill-conditioned matrices $H_m \in \mathbb{R}^{m \times m}$, $m = 1, 2, 3, \dots$, whose condition numbers get rapidly worse with increasing m . Now we can observe, at least intuitively, that the ill conditioning originates with the basis $\phi_1(t) = 1$, $\phi_2(t) = t$, $\phi_3(t) = t^2, \dots$. If you plot the functions t^k on $[0, 1]$, you will see that with increasing k they look more and more alike. That is, they come closer and closer to being linearly dependent. Thus the basis $\phi_1, \dots, \phi_m$ is (in some sense) ill conditioned and becomes increasingly ill conditioned as m is increased. This ill conditioning is inherited by the Hilbert matrices.

3.6 UPDATING THE QR DECOMPOSITION

There are numerous applications in which the data matrix is updated repeatedly. For example, in a signal processing application, each row of the matrix corresponds to measurements made at a given time. Each time new measurements come in, a new row is added. Now suppose we have solved a least-squares problem (to estimate the trajectory of an object, say) at a particular time, but now we want to solve the problem again, incorporating additional data that has just arrived. To solve the first problem we computed a QR decomposition. Must we now compute a new QR decomposition from scratch, or can we somehow update the old one? It turns out that we can update the old QR decomposition much more cheaply than we can compute a new one from scratch.

Adding a Row

Suppose $A \in \mathbb{R}^{n \times m}$, $n \geq m$, $\text{rank}(A) = m$, and we have a decomposition $A = QR$, where $Q \in \mathbb{R}^{n \times n}$ is orthogonal, and $R \in \mathbb{R}^{n \times m}$ is upper triangular. That is, the upper $m \times m$ block of R is upper triangular, and the rest of R is zero. Now let

$$\tilde{A} = \begin{bmatrix} A_1 \\ z^T \\ A_2 \end{bmatrix} \in \mathbb{R}^{n+1 \times m}, \text{ where } A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}, \text{ and } z^T \text{ is a new row, which we wish}$$

to insert somewhere among the rows of A . (It might seem simpler just to adjoin z^T at the bottom of the matrix A , but, as we shall show, it is no harder to insert it anywhere in the matrix.) Let us figure out how to derive the QR decomposition $\tilde{A} = \tilde{Q}\tilde{R}$ from the QR decomposition of A .

We begin by partitioning Q in the same way as we have partitioned A . Then the QR decomposition takes the form

$$\begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = \begin{bmatrix} Q_1 \\ Q_2 \end{bmatrix} R.$$

It follows immediately that

$$\tilde{A} = \begin{bmatrix} A_1 \\ z^T \\ A_2 \end{bmatrix} = \begin{bmatrix} 0 & Q_1 \\ 1 & 0^T \\ 0 & Q_2 \end{bmatrix} \begin{bmatrix} z^T \\ R \end{bmatrix}. \quad (3.6.1)$$

This is nearly a QR decomposition. The only problem is that $\begin{bmatrix} z^T \\ R \end{bmatrix}$ is Hessenberg, not triangular. We just need to transform it to upper triangular form. We have

$$\begin{bmatrix} z \\ R \end{bmatrix} = \begin{bmatrix} z_1 & z_2 & \cdots \\ r_{11} & r_{12} & \cdots \\ 0 & r_{22} & \cdots \\ 0 & 0 & \ddots \\ \vdots & \vdots & \end{bmatrix}.$$

There is a plane rotator U_1^T acting on rows 1 and 2 such that $U_1^T \begin{bmatrix} z^T \\ R \end{bmatrix}$ has a zero where r_{11} was. This transformation obviously does not destroy any of the zeros that were already present in the matrix. Once we have annihilated r_{11} , we can annihilate r_{22} by applying a plane rotator U_2^T that acts on rows 2 and 3. This does not disturb any zeros that were already present. In particular, the zero in the r_{11} position stays zero because $r_{21} = 0$. The next step is to apply a rotator U_3^T , acting on rows 3 and 4 that annihilates r_{33} . If we continue in this manner for m steps, we obtain an upper triangular matrix

$$\tilde{R} = U_m^T \cdots U_2^T U_1^T \begin{bmatrix} z^T \\ R \end{bmatrix} \in \mathbb{R}^{(n+1) \times m}.$$

Letting

$$\tilde{Q} = \begin{bmatrix} 0 & Q_1 \\ 1 & 0^T \\ 0 & Q_2 \end{bmatrix} U_1 U_2 \cdots U_m,$$

we have $\tilde{A} = \tilde{Q}\tilde{R}$, with $\tilde{Q} \in \mathbb{R}^{(n+1) \times (n+1)}$ orthogonal and $\tilde{R} \in \mathbb{R}^{(n+1) \times m}$ upper triangular.

The cost of producing $\tilde{R}$ is quite low. There are m plane rotators, each acting on rows of effective length m or less. Thus the flop count is clearly $O(m^2)$. You can easily check that it is $3m^2 + O(m)$, to be more precise. Depending on the application, we may or may not wish to update Q . If we do wish to make that update and we have Q stored explicitly, we can obtain $\tilde{Q}$ by applying m plane rotators on the right. This costs $6nm + O(m)$ flops. Recalling that the cost of a QR decomposition from scratch is around $2nm^2$ flops, we see that the update is much cheaper, especially the update of R .

Exercise 3.6.2

- (a) Show that about $3m^2$ flops are needed to produce $\tilde{R}$.
- (b) Show that $\tilde{Q}$ can be obtained from Q in about $6nm$ flops if Q is stored explicitly.

□

Exercise 3.6.3 The heart of the procedure that we have just outlined is to reduce the matrix

$$\begin{bmatrix} z^T \\ R \end{bmatrix}$$

to upper triangular form. Reformulate the algorithm so that the matrix that is to be reduced to triangular form is

$$\begin{bmatrix} R \\ z^T \end{bmatrix}.$$

Show how to return this matrix to upper triangular form by a sequence of m plane rotators.

□

Exercise 3.6.4 Suppose we have a condensed QR decomposition $A = \hat{Q}\hat{R}$, where $\hat{Q} \in \mathbb{R}^{n \times m}$ is isometric and $\hat{R} \in \mathbb{R}^{m \times m}$ is upper triangular. Develop a procedure for updating this decomposition when a row is adjoined to A . Make sure you get the dimensions of the matrices right. (You'll have to throw something away in the end.)

□

The procedure we have just outlined also serves as a method for updating the Cholesky decomposition. If we use the normal equations to solve the least-squares problem, we need the Cholesky decomposition of $A^T A$, i.e. $A^T A = \hat{R}^T \hat{R}$. If we now add a row to A , we need the Cholesky decomposition of the updated matrix $\tilde{A}^T \tilde{A}$. It is easy to show that the Cholesky factor $\tilde{R}$ is the same as the matrix $\hat{R}$ in the decomposition $A = \hat{Q}\hat{R}$ (cf. proof of Theorem 3.2.46 and Exercise 3.5.28). The updating procedure that we have outlined above (and, in particular, in Exercise 3.6.4), does not require knowledge of the orthogonal matrix Q (or $\hat{Q}$), so it can be used to update the Cholesky factor in $3m^2$ flops.

Exercise 3.6.5 Let $M \in \mathbb{R}^{m \times m}$ be a positive definite matrix, and let R be its Cholesky factor. Let $z \in \mathbb{R}^m$, and let $\tilde{M} = M + zz^T$. Since the matrix zz^T has rank one, $\tilde{M}$ is called a *rank-one update* of M .

- (a) Show that $\tilde{M}$ is positive definite.
- (b) Let $\tilde{R}$ be the Cholesky factor of $\tilde{M}$. Outline a procedure for obtaining $\tilde{R}$ from R and z in $O(m^2)$ flops.

□

Deleting a Row

If we are constantly adding new rows of data to a matrix, we might also sometimes wish to delete old, "out-of-date" rows. We thus consider the problem of updating the QR decomposition when we have deleted a row. This procedure is commonly known

as *downdating*. Suppose we have the decomposition $\tilde{A} = \tilde{Q}\tilde{R}$, where $\tilde{A} = \begin{bmatrix} A_1 \\ z^T \\ A_2 \end{bmatrix}$,

and we seek the QR decomposition of $A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$. We can essentially reverse the process we used for adding a row. However, the reversed process requires knowledge of $\tilde{Q}$ and generally requires more work. A downdating procedure that acts only on $\tilde{R}$ and makes no use of $\tilde{Q}$ is outlined in Exercises 3.6.12 through 3.6.15.

The objective is to transform $\tilde{Q}$ essentially to the form

$$\begin{bmatrix} 0 & Q_1 \\ 1 & 0^T \\ 0 & Q_2 \end{bmatrix},$$

as in (3.6.1). Let

$$\tilde{Q} = \begin{bmatrix} \tilde{Q}_1 \\ w^T \\ \tilde{Q}_2 \end{bmatrix},$$

where the row w^T is in the same position as the row that is to be removed from $\tilde{A}$. We need to transform w^T to the form $[\gamma \ 0 \ \cdots \ 0]$ (where $|\gamma| = 1$). Let U_n be a plane rotator acting on positions n and $n+1$ such that $w^T U_n$ has a zero in its last $((n+1)$ st) position. Then let U_{n-1} be a rotator acting on positions $n-1$ and n such that $w^T U_n U_{n-1}$ has zeros in its last two positions. Continuing in this way we produce plane rotators $U_{n-2}, \dots, U_1$ such that $w^T U_n U_{n-1} \cdots U_1$ has the form $[\gamma \ 0 \ \cdots \ 0]$. If we apply this sequence of rotators to $\tilde{Q}$, we obtain a matrix $\tilde{Q} = \tilde{Q} U_n \cdots U_1$ of the form

$$\tilde{Q} = \begin{bmatrix} 0 & Q_1 \\ \gamma & 0^T \\ 0 & Q_2 \end{bmatrix}. \quad (3.6.6)$$

The first column, apart from the γ , is necessarily zero because the matrix is orthogonal and therefore has orthonormal rows.

Let $\tilde{R} = U_1^T \cdots U_n^T \tilde{R}$. Then $\tilde{A} = \tilde{Q}\tilde{R} = (\tilde{Q} U_n \cdots U_1)(U_1^T \cdots U_n^T \tilde{R}) = \tilde{Q}\tilde{R}$, so let us examine $\tilde{R}$. $\tilde{R}$ is an $(n+1) \times m$ matrix whose top m rows form an upper triangular matrix and whose last $n+1-m$ rows are zero. Now consider how $\tilde{R}$ is altered when we apply the plane rotators $U_n^T, U_{n-1}^T, \dots, U_1^T$. The first rotators, $U_n^T, \dots, U_{m+1}^T$, have no effect at all on $\tilde{R}$, because they operate on rows that are identically zero. The first to have any effect is U_m^T , which acts on rows m and $m+1$. Row m has one nonzero element, namely $\tilde{r}_{mm}$. When U_m^T is applied, this nonzero entry is combined with the zero in position $(m+1, m)$ to produce a nonzero entry in

that position. This is clearly the only new nonzero entry produced by this operation. Similarly, application of U_{m-1}^T acts on rows $m-1$ and m and creates a new nonzero entry in position $(m, m-1)$ (and nowhere else), because $\tilde{r}_{m-1, m-1}$ is nonzero. The pattern is now clear. Application of $U_m^T, \dots, U_1^T$ produces new nonzero entries in positions $(m+1, m), (m, m-1), \dots, (2, 1)$, and nowhere else. This means that $\tilde{R}$ has upper Hessenberg form. Thus we can partition $\tilde{R}$ as

$$\tilde{R} = \begin{bmatrix} v^T \\ R \end{bmatrix},$$

where v^T is the first row of $\tilde{R}$, and R is upper triangular. Combining this with (3.6.6), we have

$$\begin{bmatrix} A_1 \\ z^T \\ A_2 \end{bmatrix} = \tilde{A} = \tilde{Q}\tilde{R} = \begin{bmatrix} 0 & Q_1 \\ \gamma & 0^T \\ 0 & Q_2 \end{bmatrix} \begin{bmatrix} v^T \\ R \end{bmatrix}.$$

Focusing on the top block, we have $A_1 = 0v^T + Q_1R = Q_1R$. Now looking at the bottom block, we have $A_2 = 0v^T + Q_2R$. Putting these blocks together, leaving out the middle row, and letting $Q = \begin{bmatrix} Q_1 \\ Q_2 \end{bmatrix}$, we obtain

$$A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} = \begin{bmatrix} Q_1 \\ Q_2 \end{bmatrix} R = QR.$$

Since the rows of Q are orthonormal, Q is an orthogonal matrix. Thus we now have the QR decomposition of A .

Exercise 3.6.7 How much does this downdating procedure cost ...

(a) ... if only R is wanted?

(b) ... if Q is wanted?

□

Exercise 3.6.8 Why does the downdating procedure take more work than the updating procedure? □

Since this procedure requires knowledge of the orthogonal matrix $\tilde{Q}$, we cannot use it for downdating the Cholesky decomposition $\tilde{A}^T \tilde{A} = \tilde{R}^T \tilde{R}$, when a row is removed from $\tilde{A}$. Fortunately a procedure for downdating the Cholesky decomposition does exist. It is quite economical ($O(m^2)$), but it is more delicate, involving the use of hyperbolic transformations. See, for example, [33, §12.5.4] and Exercises 3.6.12 through 3.6.15.

Adding a Column

Consider an experiment in which a large number of individuals, e.g. fruit flies or guinea pigs, are measured in various ways, e.g. wingspan, weight, length of ears. If m characteristics are measured for each of n individuals, the data can be assembled in an $n \times m$ matrix A in which each row corresponds to an individual and each column corresponds to a characteristic. We may be able to learn something about our population by solving some least squares problems using this data. If we want to add a characteristic to our study, we must add a column to the matrix A . Suppose we have a QR decomposition of A . If we adjoin a column to A , can we obtain the QR decomposition of the new matrix efficiently by updating the QR decomposition of A ?

Let $A = [A_1 \ A_2]$ and $\tilde{A} = [A_1 \ z \ A_2]$, where z is the column that is to be adjoined. We have a decomposition $A = QR$, with $Q \in \mathbb{R}^{n \times n}$ orthogonal and $R \in \mathbb{R}^{n \times m}$ upper triangular. Partitioning R conformably with A , we have

$$\begin{bmatrix} A_1 & A_2 \end{bmatrix} = Q \begin{bmatrix} R_1 & R_2 \end{bmatrix}.$$

Let us see how to update this decomposition to obtain the QR decomposition of $\tilde{A}$. If we let $w = Q^T z = Q^{-1} z$, then

$$\tilde{A} = \begin{bmatrix} A_1 & z & A_2 \end{bmatrix} = Q \begin{bmatrix} R_1 & w & R_2 \end{bmatrix}.$$

We have disturbed the triangular form of R ; our task is to restore it. Suppose w is the k th column. To obtain triangular form we must annihilate the bottom $n - k$ entries of w . Let U_n be a plane rotator acting on rows n and $n - 1$ such that $U_n w$ has a zero in the n th position. Let U_{n-1} be a plane rotator on rows $n - 1$ and $n - 2$ such that $U_{n-1} U_n w$ has a zero in position $n - 1$. The zero that was created on the previous step remains untouched. Continuing in this fashion, we can produce plane rotators $U_{n-2}, \dots, U_{k+1}$ such that $U_{k+1} \cdots U_n w$ has zeros in positions $k + 1, \dots, n$. Let

$$\tilde{R} = U_{k+1} \cdots U_n \begin{bmatrix} R_1 & w & R_2 \end{bmatrix}.$$

It is easy to check that $\tilde{R}$ has upper triangular form. The rotators do not alter R_1 at all. They do affect R_2 , but only to the extent that they fill out $\tilde{R}$ to a full upper-triangular matrix. Let $\tilde{Q} = QU_n^T \cdots U_{k+1}^T$. Then $\tilde{A} = \tilde{Q} \tilde{R}$. This is our desired decomposition.

Exercise 3.6.9 Verify that $\tilde{R}$ is an upper-triangular matrix. □

Unfortunately this updating procedure is not particularly economical. The operation $w = Q^T z$ already costs $2n^2$ flops, and it requires knowledge of Q . If $n \geq m^2$, it is not worthwhile. The MATLAB command `qrinsert` implements this procedure.

Deleting a Column

Now suppose we wish to delete a characteristic from our sample. This corresponds to deleting a column from A . How do we perform the update, or perhaps we should

say *downdate*, in this case? Fortunately it turns out that removing a column is much cheaper than adding one.

Let $\tilde{A} = [A_1 \ z \ A_2]$ and $A = [A_1 \ A_2]$. Suppose we have the decomposition $\tilde{A} = \tilde{Q}\tilde{R}$, and we would like the QR decomposition of A . Let $\tilde{R} = [\tilde{R}_1 \ w \ \tilde{R}_2]$. Removing a column from the equation $\tilde{A} = \tilde{Q}\tilde{R}$, we obtain

$$A = [A_1 \ A_2] = \tilde{Q} [\tilde{R}_1 \ \tilde{R}_2]. \quad (3.6.10)$$

This is not quite a QR decomposition, because the “ R ” matrix is not upper triangular. For example, if we remove the third column from a 7×5 upper triangular matrix, we obtain a matrix of the form

$$\begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

It is upper Hessenberg but can be reduced to triangular form by two rotators, U_3^T , acting on rows 3 and 4, followed by U_5^T , acting on rows 4 and 5. More generally, in (3.6.10), if the row w that was removed from $\tilde{R}$ was the k th row, $m - k + 1$ plane rotators, $U_k^T, \dots, U_m^T$ will be needed to return the matrix to triangular form. Letting $R = U_m^T \cdots U_k^T [\tilde{R}_1 \ \tilde{R}_2]$ and $Q = \tilde{Q}U_k \cdots U_m$, we have our QR decomposition of A .

This procedure can be carried out without knowledge of the orthogonal matrix $\tilde{Q}$, so it also serves as a means of *downdating* the Cholesky decomposition of $\tilde{A}^T \tilde{A}$, when a column is removed from $\tilde{A}$. The cost of obtaining R from $\tilde{R}$ depends on which column is removed. In the worst case (first column is removed) m rotators are needed, and they operate on columns of effective length m or less. Therefore the flop count is $O(m^2)$. In the best case (last column is removed) no flops are needed.

This procedure is implemented in the MATLAB command `qrdelete`.

Exercise 3.6.11

- Suppose the k th column is deleted from $\tilde{A}$. Show that the cost of producing R is about $3(m - k)^2$ flops.
- Show that the cost of computing Q from $\tilde{Q}$ is about $6n(m - k)$ flops if $\tilde{Q}$ is given explicitly.

□

Additional Exercises

Exercise 3.6.12 A 2×2 matrix H is called a *hyperbolic transformation* if it has the form

$$H = \begin{bmatrix} c & s \\ s & c \end{bmatrix},$$

where $c > 0$ and

$$c^2 - s^2 = 1. \quad (3.6.13)$$

The set of all (c, s) that satisfy (3.6.13) is a hyperbola in the c - s plane. For any pair (c, s) satisfying (3.6.13) there is a number α such that $c = \cosh \alpha$ and $s = \sinh \alpha$.

- Show that every hyperbolic transformation H is nonsingular. Find the determinant of H . What is H^{-1} ? Note that H^{-1} is also hyperbolic.
- Let $J = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$. Show that if H is hyperbolic, then $H^T J H = J$. Of course, $H = H^T$, but it turns out to be useful to write the identity in terms of the transpose.
- Show that if $\begin{bmatrix} a \\ b \end{bmatrix} \in \mathbb{R}^2$ with $|a| > |b|$, then there is a unique hyperbolic transformation H such that

$$\begin{bmatrix} c & s \\ s & c \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sqrt{a^2 - b^2} \\ 0 \end{bmatrix}.$$

Obtain formulas (resembling (3.2.11)) for c and s in terms of a and b .⁹ The condition $c^2 - s^2 = 1$ does not put any bound on c and s ; they can be arbitrarily large. (Graphically, the hyperbola is an unbounded figure.) It follows that hyperbolic transformations can be ill conditioned and lack the unconditional stability of rotators. See Exercise 3.6.16.

- We can embed hyperbolic transformations in larger matrices, just as we did for rotators. Let

$$H = \begin{bmatrix} I & & & \\ & c & & s \\ & & I & \\ & s & & c \\ & & & & I \end{bmatrix},$$

where $c > 0$ and $c^2 - s^2 = 1$. Suppose the rows and columns in which the hyperbolic transformation is embedded are i and j ($i < j$). Let J be

⁹If $|a| = |b|$, there is no hyperbolic transformation that can transform b to zero. If $|a| < |b|$, a different type of hyperbolic transformation can do the job. Since we will not need that type of transformation, we do not introduce it here.

any diagonal matrix with the entries 1 and -1 in positions (i, i) and (j, j) , respectively. Show that $H^T J H = J$. (Again the transpose is unnecessary. However, this identity also holds for products of matrices of this type, and then the transpose really is needed.) Show that if $\hat{S} = HS$, then $\hat{S}^T J \hat{S} = S^T J S$. Show that $\hat{S}$ and S differ only in the i th and j th rows.

□

Exercise 3.6.14 In this exercise we show how to use hyperbolic transformations to downdate the Cholesky decomposition. Let

$$\tilde{A} = \begin{bmatrix} A_1 \\ z^T \\ A_2 \end{bmatrix} \in \mathbb{R}^{(n+1) \times m} \quad \text{and} \quad A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} \in \mathbb{R}^{n \times m},$$

and suppose A has rank m . Suppose we have $\tilde{R} \in \mathbb{R}^{m \times m}$, the Cholesky factor of $\tilde{A}^T \tilde{A}$, and we would like to obtain $R \in \mathbb{R}^{m \times m}$, the Cholesky factor of $A^T A$.

(a) Let

$$J = \begin{bmatrix} I_m & 0 \\ 0^T & -1 \end{bmatrix} \in \mathbb{R}^{(m+1) \times (m+1)}.$$

Show that

$$A^T A = \tilde{A}^T \tilde{A} - z z^T = \tilde{R}^T \tilde{R} - z z^T = \begin{bmatrix} \tilde{R}^T & z \end{bmatrix} J \begin{bmatrix} \tilde{R} \\ z^T \end{bmatrix}.$$

Our plan is to obtain the Cholesky factor of $A^T A$ by annihilating the entries of z^T in the matrix

$$\tilde{S} = \begin{bmatrix} \tilde{R} \\ z^T \end{bmatrix} = \begin{bmatrix} \tilde{r}_{11} & \tilde{r}_{12} & \cdots \\ 0 & \tilde{r}_{22} & \cdots \\ 0 & 0 & \ddots \\ \vdots & \vdots & \vdots \\ z_1 & z_2 & \cdots \end{bmatrix}.$$

For this purpose we will use hyperbolic transformations.

(b) Using the fact that $A^T A$ is positive definite, demonstrate that $\tilde{r}_{11}^2 - z_1^2 > 0$, whence $|\tilde{r}_{11}| > |z_1|$. Construct a hyperbolic transformation $H_1 \in \mathbb{R}^{(m+1) \times (m+1)}$ that acts on rows 1 and $m+1$, such that

$$S_1 = H_1 \tilde{S} = \begin{bmatrix} r_{11} & r_{12} & \cdots \\ 0 & \tilde{r}_{22} & \cdots \\ 0 & 0 & \ddots \\ \vdots & \vdots & \vdots \\ 0 & \hat{z}_2 & \cdots \end{bmatrix}.$$

The entry z_1 has been annihilated. The entry $\hat{z}_2$ differs from z_2 , but it is, in general, nonzero.

- (c) Show that $A^T A = \tilde{S}^T J \tilde{S} = S_1^T J S_1$.
- (d) In Exercise 3.6.15 we will show that $|\tilde{r}_{22}| > |\hat{z}_2|$, and appropriate corresponding inequalities hold at all subsequent steps. Assuming these inequalities to be true, sketch an algorithm that applies a sequence of m hyperbolic transformations $H_1, \dots, H_m$ to $\tilde{S}$, thereby transforming it to a matrix $S = S_m \cdots H_1 \tilde{S}$ of the form $S = \begin{bmatrix} R \\ 0^T \end{bmatrix}$, where R is upper triangular and has positive main-diagonal entries. Show that $A^T A = S^T J S = R^T R$. Thus R is the Cholesky factor of $A^T A$.
- (d) Show that this downdating procedure requires about $3m^2$ flops in all.

□

Exercise 3.6.15 After k steps of the algorithm sketched in the previous exercise, we have transformed $\tilde{S}$ to the form

$$S_k = \begin{bmatrix} R_{11} & R_{12} \\ 0 & \tilde{R}_{22} \\ 0^T & \hat{z}^T \end{bmatrix},$$

where R_{11} is $k \times k$. We have $A^T A = S_k^T J S_k$, where J is as defined in the previous exercise. Let $A = \begin{bmatrix} A^{(1)} & A^{(2)} \end{bmatrix}$, where $A^{(1)}$ has k columns.

- (a) Show that $A^{(1)T} A^{(1)} = R_{11}^T R_{11}$ and $A^{(1)T} A^{(2)} = R_{11}^T R_{12}$, and deduce that R_{11} and R_{12} are blocks of the Cholesky factor of $A^T A$.
- (b) Show that $A^{(2)T} A^{(2)} - R_{12}^T R_{12} = \tilde{R}_{22}^T \tilde{R}_{22} - \hat{z} \hat{z}^T$. Show that the matrix $A^{(2)T} A^{(2)} - R_{12}^T R_{12}$ is the Schur complement of $A^{(1)T} A^{(1)}$ in $A^T A$ and is positive definite. Here we essentially are retreading ground covered in Section 1.4, especially Exercise 1.4.58 and surrounding material.
- (c) Use the positive definiteness of $A^{(2)T} A^{(2)} - R_{12}^T R_{12}$ to prove that the $(1, 1)$ entry of $\tilde{R}_{22}$, which we call $\tilde{r}_{k+1, k+1}$ and the leading entry of $\hat{z}^T$, which we call $\hat{z}_{k+1}$, satisfy $|\tilde{r}_{k+1, k+1}| > |\hat{z}_{k+1}|$. Thus we can set up the hyperbolic rotator H_{k+1} for the $(k+1)$ st step.

□

Exercise 3.6.16 In this exercise we show that hyperbolic transformations can be arbitrarily ill conditioned.

- (a) Let L be a huge positive number (as huge as you please). Find two positive numbers c and s such that $c \geq L$, $s \geq L$, and $c^2 - s^2 = 1$. Then

$$H = \begin{bmatrix} c & s \\ s & c \end{bmatrix}$$

is a hyperbolic transformation with large norm. Show that $\|H\|_\infty = \|H^{-1}\|_\infty = c + s \geq 2L$ and $\kappa_\infty(H) \geq 4L^2$.

- (b) Let $v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ and $w = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Show that $Hv = (c + s)v$ and $Hw = (c - s)w$. The vectors v and w are special vectors called *eigenvectors* of H . We will study eigenvectors in Chapters 5 and 6. Show that $H^{-1}w = (c + s)w$.
- (c) Using results of part (b), show that $\|H\|_2 \geq (c + s)$, $\|H^{-1}\|_2 \geq (c + s)$, and $\kappa_2(H) \geq (c + s)^2$. In Chapter 5 we will find that all three of these inequalities are in fact equations.
- (d) Let H be the hyperbolic transformation the maps

$$\begin{bmatrix} a \\ b \end{bmatrix} \quad \text{to} \quad \begin{bmatrix} \sqrt{a^2 - b^2} \\ 0 \end{bmatrix},$$

where $|a| > |b|$. Under what conditions on a and b will H be ill conditioned?

□

In light of the fact that hyperbolic transformations can be ill conditioned, we ought to be suspicious about the stability of any algorithm that uses them. It turns out that the downdating procedure used in Exercise 3.6.14 works well as long as $A^T A$ is “safely” positive definite, meaning well conditioned. However, if we do something reckless, such as removing so many rows that the rank of A becomes less than m , we can expect the algorithm to fail.

This page intentionally left blank

4

The Singular Value Decomposition (SVD)

The QR decomposition is a fine tool for solving least squares problems when the coefficient matrix is known to have full rank. However, if the matrix does not have full rank, or if the rank is unknown, a more powerful tool is needed. One such tool is the QR decomposition with column pivoting, which we discussed in Section 3.3. In this chapter we introduce an even more powerful tool, the singular value decomposition (SVD). This may be the most important matrix decomposition of all, for both theoretical and computational purposes.

We begin the chapter by introducing the SVD and showing that it can take a variety of forms. Then, in Section 4.2, we establish the connection between singular values and the norm and condition number of a matrix. We also show how to use the SVD to detect the (numerical) rank of matrix in the presence of roundoff errors and other uncertainties in the data, we show that rank-deficient matrices are in some sense scarce, and we show how to compute the distance to the nearest rank-deficient matrix. In Section 4.3 we show how to use the SVD to solve least squares problems, even if the coefficient matrix does not have full rank. We also introduce the pseudoinverse, an interesting generalization of the inverse of a matrix. Finally, in Section 4.4, we analyze the sensitivity of the least squares problem in the full-rank case, making use of results proved in Section 4.2.

This chapter says nothing about how to compute the SVD. That will have to wait until we have established the connection between singular values and eigenvalues.

We will continue to focus on real matrices. However, all of the developments of this chapter can be extended to complex matrices in a straightforward way.

4.1 INTRODUCTION

Let $A \in \mathbb{R}^{n \times m}$, where n and m are positive integers. We make no assumption about which of n and m is larger. Recall that the *range* of A is the subspace of $\mathbb{R}^n$ defined by $\mathcal{R}(A) = \{Ax \mid x \in \mathbb{R}^m\}$. The *rank* of A is the dimension of $\mathcal{R}(A)$.

Theorem 4.1.1 (SVD Theorem) *Let $A \in \mathbb{R}^{n \times m}$ be a nonzero matrix with rank r . Then A can be expressed as a product*

$$A = U\Sigma V^T, \quad (4.1.2)$$

where $U \in \mathbb{R}^{n \times n}$ and $V \in \mathbb{R}^{m \times m}$ are orthogonal, and $\Sigma \in \mathbb{R}^{n \times m}$ is a nonsquare “diagonal” matrix

$$\Sigma = \begin{bmatrix} \sigma_1 & & & & & \\ & \sigma_2 & & & & \\ & & \ddots & & & \\ & & & \sigma_r & & \\ & & & & 0 & \\ & & & & & \ddots \end{bmatrix} \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

To prove this theorem, work Exercise 4.1.17. The decomposition (4.1.2) is called the *singular value decomposition* of A . We will usually use the abbreviation SVD. The discussion of many aspects of the SVD will have to be deferred until after we have discussed eigenvalues and eigenvectors. For example, Theorem 4.1.1 says nothing about the uniqueness of the decomposition. It turns out that U and V are not uniquely determined, but they have some partial uniqueness properties that we will be able to discuss later. The entries $\sigma_1, \dots, \sigma_r$ of Σ are uniquely determined, and they are called the *singular values* of A . The columns of U are orthonormal vectors called *right singular vectors* of A , and the columns of V are called *left singular vectors* for reasons that will become apparent. The transpose of A has the SVD $A^T = V\Sigma^T U^T$.

Once we have discussed eigenvalues and eigenvectors, we will be able to provide a second proof of the SVD theorem, and we will develop algorithms for computing the SVD. See Section 5.9

Two matrices $A, B \in \mathbb{R}^{n \times m}$ are said to be *equivalent* if there exist nonsingular matrices $X \in \mathbb{R}^{n \times n}$ and $Y \in \mathbb{R}^{m \times m}$ such that $A = XBY$. If the matrices X and Y are orthogonal, then A and B are said to be *orthogonally equivalent*. Theorem 4.1.1 shows that every $A \in \mathbb{R}^{n \times m}$ is orthogonally equivalent to a diagonal matrix.

Other Forms of the SVD Theorem

The SVD has a simple geometric interpretation, which is a consequence of the following restatement of the SVD theorem.

Theorem 4.1.3 (*Geometric SVD Theorem*) Let $A \in \mathbb{R}^{n \times m}$ be a nonzero matrix with rank r . Then $\mathbb{R}^m$ has an orthonormal basis $v_1, \dots, v_m$, $\mathbb{R}^n$ has an orthonormal basis $u_1, \dots, u_n$, and there exist $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ such that

$$Av_i = \begin{cases} \sigma_i u_i & i = 1, \dots, r \\ 0 & i = r+1, \dots, m \end{cases} \quad A^T u_i = \begin{cases} \sigma_i v_i & i = 1, \dots, r \\ 0 & i = r+1, \dots, n. \end{cases} \quad (4.1.4)$$

Exercise 4.1.5 Show that all of the equations (4.1.4) follow from the SVD $A = U\Sigma V^T$ (and its transpose), where $u_1, \dots, u_n$ and $v_1, \dots, v_m$ are the columns of U and V , respectively. $\square$

Exercise 4.1.6 Show that the equations in the left half of (4.1.4) imply the matrix equation $AV = U\Sigma$, the equations in the right half of (4.1.4) imply $A^T U = V\Sigma^T$, and either one of these implies the SVD $A = U\Sigma V^T$. $\square$

Think of A as a linear transformation that maps vectors $x \in \mathbb{R}^m$ to vectors $Ax \in \mathbb{R}^n$. Theorem 4.1.3 shows that $\mathbb{R}^m$ and $\mathbb{R}^n$ have orthonormal bases such that A maps the i th basis vector in $\mathbb{R}^m$ to a multiple of the i th basis vector in $\mathbb{R}^n$ (and A^T acts similarly). From elementary linear algebra we know that if we choose bases in $\mathbb{R}^m$ and $\mathbb{R}^n$, then we can represent this or any linear transformation by a matrix with respect to these bases. Theorem 4.1.3 says simply that the diagonal matrix Σ is the matrix of the transformation A with respect to the orthonormal bases $v_1, \dots, v_m$ and $u_1, \dots, u_n$ (and Σ^T is the matrix of the transformation A^T). The action of A is depicted in a simple way by the following diagram.

$$\begin{array}{ccc} & \xrightarrow{A} & \\ v_1 & \xrightarrow{\sigma_1} & u_1 \\ v_2 & \xrightarrow{\sigma_2} & u_2 \\ \vdots & & \vdots \\ v_r & \xrightarrow{\sigma_r} & u_r \\ v_{r+1} & \left. \begin{array}{c} \vdots \\ v_m \end{array} \right\} \rightarrow & 0 \end{array}$$

A similar diagram holds for A^T . If we set the two diagrams side by side, we obtain

$$\begin{array}{ccccc} & \xrightarrow{A} & & \xrightarrow{A^T} & \\ v_1 & \xrightarrow{\sigma_1} & u_1 & \xrightarrow{\sigma_1} & v_1 \\ v_2 & \xrightarrow{\sigma_2} & u_2 & \xrightarrow{\sigma_2} & v_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ v_r & \xrightarrow{\sigma_r} & u_r & \xrightarrow{\sigma_r} & v_r \\ v_{r+1} & \left. \begin{array}{c} \vdots \\ v_m \end{array} \right\} \rightarrow & 0 & \left. \begin{array}{c} u_{r+1} \\ \vdots \\ u_n \end{array} \right\} \rightarrow & 0 \end{array} \quad (4.1.7)$$

which serves as a pictorial representation of the SVD Theorem.

The SVD displays orthonormal bases for the four fundamental subspaces $\mathcal{R}(A)$, $\mathcal{N}(A)$, $\mathcal{R}(A^T)$, and $\mathcal{N}(A^T)$. It is clear from (4.1.7) that

$$\begin{aligned}\mathcal{R}(A) &= \text{span}\{u_1, \dots, u_r\} \\ \mathcal{N}(A) &= \text{span}\{u_{r+1}, \dots, u_m\} \\ \mathcal{R}(A^T) &= \text{span}\{v_1, \dots, v_r\} \\ \mathcal{N}(A^T) &= \text{span}\{v_{r+1}, \dots, v_n\}.\end{aligned}\tag{4.1.8}$$

From these representations we see that $\mathcal{R}(A^T) = \mathcal{N}(A)^\perp$ and $\mathcal{R}(A) = \mathcal{N}(A^T)^\perp$; we proved these equations by other means in Theorem 3.5.10.

Another immediate consequence of (4.1.8) is the following fact, which can also be deduced by more elementary means. We record it here for future reference.

Corollary 4.1.9 *Let $A \in \mathbb{R}^{n \times m}$. Then $\dim(\mathcal{R}(A)) + \dim(\mathcal{N}(A)) = m$.*

The next theorem gives a more condensed version of the SVD.

Theorem 4.1.10 (Condensed SVD Theorem) *Let $A \in \mathbb{R}^{n \times m}$ be a nonzero matrix of rank r . Then there exist $\hat{U} \in \mathbb{R}^{n \times r}$, $\hat{\Sigma} \in \mathbb{R}^{r \times r}$, and $\hat{V} \in \mathbb{R}^{m \times r}$ such that $\hat{U}$ and $\hat{V}$ are isometries, $\hat{\Sigma}$ is a diagonal matrix with main-diagonal entries $\sigma_1 \geq \dots \geq \sigma_r > 0$, and*

$$A = \hat{U} \hat{\Sigma} \hat{V}^T.$$

Exercise 4.1.11 Prove Theorem 4.1.10 by writing the SVD $A = U \Sigma V^T$ block form in an appropriate way and then chopping off the unneeded blocks. $\square$

Finally we present one more useful form of the SVD.

Theorem 4.1.12 *Let $A \in \mathbb{R}^{n \times m}$ be a nonzero matrix with rank r . Let $\sigma_1, \dots, \sigma_r$ be the singular values of A , with associated right and left singular vectors $v_1, \dots, v_r$ and $u_1, \dots, u_r$, respectively. Then*

$$A = \sum_{j=1}^r \sigma_j u_j v_j^T.$$

Exercise 4.1.13 Let $B = \sum_{j=1}^r \sigma_j u_j v_j^T$. Prove Theorem 4.1.12 by showing that $B = A$. (Show that $A v_i = B v_i$ for $i = 1, \dots, m$.) $\square$

Exercise 4.1.14 Obtain a second proof of Theorem 4.1.12 by partitioning the decomposition $A = \hat{U} \hat{\Sigma} \hat{V}^T$ from Theorem 4.1.10 in an appropriate way. $\square$

Exercise 4.1.15 Let

$$A = \begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \end{bmatrix}.$$

By inspection, write down the SVD in the condensed form of Theorem 4.1.10 and in the form given by Theorem 4.1.12. How do the two compare? $\square$

Exercise 4.1.16 In MATLAB you can use the command `svd` to compute either the singular values or the singular value decomposition of a matrix. Type `help svd` to find out how to use this command. Use MATLAB's `svd` command to check your result from the previous exercise. $\square$

Exercise 4.1.17 In this exercise you will prove Theorem 4.1.1 by induction on r , the rank of A .

- (a) Suppose $A \in \mathbb{R}^{n \times m}$ has rank 1. Let $u_1 \in \mathbb{R}^n$ be a vector in $\mathcal{R}(A)$ such that $\|u_1\|_2 = 1$. Show that every column of A is a multiple of u_1 . Show that A can be written in the form $A = \sigma_1 u_1 v_1^T$, where $v_1 \in \mathbb{R}^m$, $\|v_1\|_2 = 1$, and $\sigma_1 > 0$.
- (b) Continuing from part (a), demonstrate that there is an orthogonal matrix $U \in \mathbb{R}^{n \times n}$ whose first column is u_1 . (For example, U can be a reflector that maps the unit vector e_1 to u_1 .) Similarly there is an orthogonal $V \in \mathbb{R}^{m \times m}$ whose first column is v_1 . Show that $A = U\Sigma V^T$, where $\Sigma \in \mathbb{R}^{n \times m}$ has only one nonzero entry, σ_1 , in position $(1, 1)$. Thus every matrix of rank 1 has an SVD.
- (c) Now suppose $A \in \mathbb{R}^{n \times m}$ has rank $r > 1$. Let v_1 be a unit vector in the direction of maximum magnification by A , i.e. $\|v_1\|_2 = 1$, and $\|Av_1\|_2 = \max_{\|u\|_2=1} \|Au\|_2$. Let $\sigma_1 = \|Av_1\|_2 = \|A\|_2$, and let $u_1 = \sigma_1^{-1} Av_1$. Let $\tilde{U} \in \mathbb{R}^{n \times n}$ and $\tilde{V} \in \mathbb{R}^{m \times m}$ be orthogonal matrices with first column u_1 and v_1 , respectively. Let $\tilde{A} = \tilde{U}^T A \tilde{V}$, so that $A = \tilde{U} \tilde{A} \tilde{V}^T$. Show that $\tilde{A}$ has the form

$$\tilde{A} = \begin{bmatrix} \sigma_1 & z^T \\ 0 & \hat{A} \end{bmatrix}, \quad (4.1.18)$$

where $z \in \mathbb{R}^{m-1}$ and $\hat{A} \in \mathbb{R}^{(n-1) \times (m-1)}$.

- (d) Show that the vector z in (4.1.18) must be zero. You may do this as follows: Let $w = \begin{bmatrix} \sigma_1 \\ z \end{bmatrix} \in \mathbb{R}^n$. Show that $\|\tilde{A}w\|_2 / \|w\|_2 \geq \sqrt{\sigma_1^2 + z^T z}$. Then show that this inequality forces $z = 0$. Thus

$$\tilde{A} = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \hat{A} \end{bmatrix}.$$

- (e) Show that $\hat{A}$ has rank $r - 1$. By the induction hypothesis $\hat{A}$ has an SVD $\hat{A} = \hat{U} \hat{\Sigma} \hat{V}^T$. Let $\sigma_2 \geq \sigma_3 \geq \dots \geq \sigma_r$ denote the positive main-diagonal entries of $\hat{\Sigma}$. Show that $\sigma_1 \geq \sigma_2$. Embed the SVD $\hat{A} = \hat{U} \hat{\Sigma} \hat{V}^T$ in larger matrices to obtain an SVD of $\tilde{A}$. Then use the equation $A = \tilde{U} \tilde{A} \tilde{V}^T$ to obtain an SVD of A .

$\square$

4.2 SOME BASIC APPLICATIONS OF SINGULAR VALUES

Relationship to Norm and Condition Number

In Section 2.1 we defined the *spectral* norm to be the norm induced by the Euclidean vector norm:

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}.$$

The discussion in Section 2.1 was restricted to square matrices, but this definition makes sense for nonsquare matrices as well. Geometrically $\|A\|_2$ represents the maximum magnification that can be undergone by any vector $x \in \mathbb{R}^m$ when acted on by A . In light of (4.1.7) (and Exercise 4.1.17), it should not be surprising that $\|A\|_2$ equals the maximum singular value of A .

Theorem 4.2.1 *Let $A \in \mathbb{R}^{n \times m}$ have singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. Then $\|A\|_2 = \sigma_1$.*

Proof. We must show that $\max_{x \neq 0} \|Ax\|_2 / \|x\|_2 = \sigma_1$. First notice that since $Av_1 = \sigma_1 u_1$,

$$\frac{\|Av_1\|_2}{\|v_1\|_2} = \sigma_1 \frac{\|u_1\|_2}{\|v_1\|_2} = \sigma_1,$$

so $\max_{x \neq 0} \|Ax\|_2 / \|x\|_2 \geq \sigma_1$. Now we must show that no other vector is magnified by more than σ_1 .

Let $x \in \mathbb{R}^m$. Then x can be expressed as a linear combination of the right singular vectors of A : $x = c_1 v_1 + c_2 v_2 + \dots + c_m v_m$. Since $v_1, \dots, v_m$ are orthonormal, $\|x\|_2^2 = |c_1|^2 + \dots + |c_m|^2$. Now $Ax = c_1 Av_1 + \dots + c_r Av_r + \dots + c_m Av_m = \sigma_1 c_1 u_1 + \dots + \sigma_r c_r u_r + 0 + \dots + 0$, where r is the rank of A . Since $u_1, \dots, u_r$ are also orthonormal, $\|Ax\|_2^2 = |\sigma_1 c_1|^2 + \dots + |\sigma_r c_r|^2$. Thus $\|Ax\|_2^2 \leq \sigma_1^2 (|c_1|^2 + \dots + |c_r|^2) \leq \sigma_1^2 \|x\|_2^2$; that is, $\|Ax\|_2 / \|x\|_2 \leq \sigma_1$. $\square$

Since A and A^T have the same singular values, we have the following corollary.

Corollary 4.2.2 $\|A\|_2 = \|A^T\|_2$.

Exercise 4.2.3 Recall that the Frobenius matrix norm is defined by

$$\|A\|_F = \left(\sum_{i=1}^n \sum_{j=1}^m |a_{ij}|^2 \right)^{1/2}.$$

Show that $\|A\|_F = (\sigma_1^2 + \sigma_2^2 + \dots + \sigma_r^2)^{1/2}$. (Hint: Show that if $B = UC$, where U is orthogonal, then $\|B\|_F = \|C\|_F$.) $\square$

Now suppose A is square, say $A \in \mathbb{R}^{n \times n}$, and nonsingular. The spectral condition number of A is defined to be

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2.$$

Let us see how $\kappa_2(A)$ can be expressed in terms of the singular values of A . Since A has rank n , it has n strictly positive singular values, and its action is described completely by the following diagram:

$$\begin{array}{ccc} & A & \\ v_1 & \xrightarrow{\sigma_1} & u_1 \\ v_2 & \xrightarrow{\sigma_2} & u_2 \\ \vdots & & \vdots \\ v_n & \xrightarrow{\sigma_n} & u_n \end{array}$$

It follows that the corresponding diagram for A^{-1} is

$$\begin{array}{ccc} & A^{-1} & \\ u_1 & \xrightarrow{\sigma_1^{-1}} & v_1 \\ u_2 & \xrightarrow{\sigma_2^{-1}} & v_2 \\ \vdots & & \vdots \\ u_n & \xrightarrow{\sigma_n^{-1}} & v_n \end{array}$$

In terms of matrices we have $A = U\Sigma V^T$ and $A^{-1} = V^{-T}\Sigma^{-1}U^{-1} = V\Sigma^{-1}U^T$. Either way we see that the singular values of A , in descending order, are $\sigma_n^{-1} \geq \sigma_{n-1}^{-1} \geq \dots \geq \sigma_1^{-1} > 0$. Applying Theorem 4.2.1 to A^{-1} , we conclude that $\|A^{-1}\|_2 = \sigma_n^{-1}$. These observations imply the following theorem.

Theorem 4.2.4 *Let $A \in \mathbb{R}^{n \times n}$ be a nonsingular matrix with singular values $\sigma_1 \geq \dots \geq \sigma_n > 0$. Then*

$$\kappa_2(A) = \frac{\sigma_1}{\sigma_n}.$$

Another expression for the condition number that was given in Chapter 2 is

$$\kappa_2(A) = \frac{\max\text{mag}(A)}{\min\text{mag}(A)},$$

where

$$\begin{aligned} \max\text{mag}(A) &= \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}, \\ \min\text{mag}(A) &= \min_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}. \end{aligned}$$

This gives a slightly different view of the condition number. From Theorem 4.2.1 we know that $\max\text{mag}(A) = \sigma_1$. It must therefore be true that $\min\text{mag}(A) = \sigma_n$.

Exercise 4.2.5 Prove that $\min\text{mag}(A) = \sigma_n$. Show that the minimum magnification is obtained by taking $x = v_n$. $\square$

In Section 3.4 we observed that the equation

$$\kappa_2(A) = \frac{\max\text{mag}(A)}{\min\text{mag}(A)} \quad (4.2.6)$$

can be used to extend the definition of κ_2 to certain nonsquare matrices. Specifically, if $A \in \mathbb{R}^{n \times m}$, $n \geq m$, and $\text{rank}(A) = m$, then $\min\text{mag}(A) > 0$, and we can take (4.2.6) as the definition of the condition number of A . If A is nonzero but does not have full rank, then (still assuming $n \geq m$) $\min\text{mag}(A) = 0$, and it is reasonable to define $\kappa_2(A) = \infty$. With this convention the following theorem holds, regardless of whether or not A has full rank.

Theorem 4.2.7 *Let $A \in \mathbb{R}^{n \times m}$, $n \geq m$ be a nonzero matrix with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$. (Here we allow some σ_i equal to zero if $\text{rank}(A) < m$.) Then $\max\text{mag}(A) = \sigma_1$, $\min\text{mag}(A) = \sigma_m$, and $\kappa_2(A) = \sigma_1/\sigma_m$.*

The proof is left as an easy exercise for you.

Exercise 4.2.8 MATLAB's command `cond` computes the condition number $\kappa_2(A)$. This works for both square and nonsquare matrices. Generate a random 3×3 matrix (`A = randn(3)`) and use MATLAB to compute $\kappa_2(A)$ three different ways: (i) using `cond`, (ii) taking the ratio of largest to smallest singular value, and (iii) computing $\|A\|_2 \|A^{-1}\|_2$ (`norm(A) * norm(inv(A))`). $\square$

The next two theorems establish other important results, which are also easy to prove.

Theorem 4.2.9 *Let $A \in \mathbb{R}^{n \times m}$ with $n \geq m$. Then $\|A^T A\|_2 = \|A\|_2^2$ and $\kappa_2(A^T A) = \kappa_2(A)^2$.*

Recall that $A^T A$ is the coefficient matrix of the normal equations (3.5.22), which can be used to solve the least-squares problem. Theorem 4.2.9 shows that the normal equations can be seriously ill conditioned even if the matrix A is only mildly ill conditioned. For example, if $\kappa_2(A) \approx 10^3$, then $\kappa_2(A^T A) \approx 10^6$.

Exercise 4.2.10

- Use the SVD of A to deduce the SVD of $A^T A$ and prove Theorem 4.2.9.
- Let $M \in \mathbb{R}^{n \times n}$ be positive definite, and let R be the Cholesky factor of M , so that $M = R^T R$. Show that $\|M\|_2 = \|R\|_2^2$ and $\kappa_2(M) = \kappa_2(R)^2$.

$\square$

The results of the next theorem will be used in the analysis of the sensitivity of the least squares problem in Section 4.4.

Theorem 4.2.11 *Let $A \in \mathbb{R}^{n \times m}$, $n \geq m$, $\text{rank}(A) = m$, with singular values $\sigma_1 \geq \dots \geq \sigma_m > 0$. Then $\|(A^T A)^{-1}\|_2 = \sigma_m^{-2}$, $\|(A^T A)^{-1} A^T\|_2 = \sigma_m^{-1}$, $\|A(A^T A)^{-1}\|_2 = \sigma_m^{-1}$, and $\|A(A^T A)^{-1} A^T\|_2 = 1$.*

The matrix $(A^T A)^{-1} A^T$ is called the *pseudoinverse* of A . $A(A^T A)^{-1}$ is the pseudoinverse of A^T . Pseudoinverses will be discussed in greater generality in the next section.

Exercise 4.2.12 Let A be as in Theorem 4.2.11 with SVD $A = U\Sigma V^T$.

- (a) Determine the singular value decompositions of the matrices

$$(A^T A)^{-1}, \quad (A^T A)^{-1} A^T, \quad A(A^T A)^{-1}, \quad \text{and} \quad A(A^T A)^{-1} A^T$$

in terms of the SVD of A . Use the orthogonality of U and V whenever possible. Pay attention to the dimensions of the various matrices.

- (b) Use the results of part (a) to prove Theorem 4.2.11.

□

Numerical Rank Determination

In the absence of roundoff errors and uncertainties in the data, the singular value decomposition reveals the rank of the matrix. Unfortunately the presence of errors makes rank determination problematic. For example, consider the matrix

$$A = \begin{bmatrix} 1/3 & 1/3 & 2/3 \\ 2/3 & 2/3 & 4/3 \\ 1/3 & 2/3 & 3/3 \\ 2/5 & 2/5 & 4/5 \\ 3/5 & 1/5 & 4/5 \end{bmatrix}. \quad (4.2.13)$$

A is obviously of rank 2, as its third column is the sum of the first two. However, if we fail to notice this relationship and decide to use, say, MATLAB to calculate its rank, we have to begin by storing the matrix in the computer. This simple act will result in roundoff errors that destroy the relationship between the columns. Technically speaking, the perturbed matrix has rank 3. If we now use MATLAB's `svd` command to compute the singular values of A , using IEEE standard double precision floating point arithmetic, we obtain

$$\sigma_1 = 2.5987 \quad \sigma_2 = 0.3682 \quad \text{and} \quad \sigma_3 = 8.6614 \times 10^{-17}.$$

Since there are three nonzero singular values, we must conclude that the matrix has rank 3. However, we cannot fail to notice that one of the singular values is tiny, on the order of the unit roundoff for IEEE double precision. Perhaps we should consider it a zero. For this reason we introduce the notion of numerical rank.

Roughly speaking, a matrix that has k “large” singular values, the others being “tiny,” has numerical rank k . For the purpose of determining which singular values are “tiny,” we need to introduce a threshold or tolerance ϵ that is roughly on the level of uncertainty in the data in the Matrix. For example, if the only errors are roundoff errors, as in the above example, we might take $\epsilon = 10u\|A\|$, where u is the unit

roundoff error. We then say that A has *numerical rank* k if A has k singular values that are substantially larger than ϵ , and all other singular values are smaller than ϵ , that is,

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k \gg \epsilon \geq \sigma_{k+1} \geq \cdots$$

MATLAB has a `rank` command, which computes the numerical rank of the matrix. When applied to the matrix A above, it yields the answer 2. MATLAB's `rank` command uses a default threshold, which can be overridden by the user. For more information type `help rank` in MATLAB.

Sometimes it is not possible to specify the numerical rank exactly. For example, imagine a 2000×1000 matrix with singular values $\sigma_j = (.9)^j$, $j = 1, \dots, 1000$. Then $\sigma_1 = .9$ and $\sigma_{1000} = 1.75 \times 10^{-46}$, so the numerical rank is definitely less than 1000. However, it is impossible to specify the numerical rank exactly, because there are no gaps in the singular values. For example

$$\begin{array}{ll} \sigma_{261} = 1.14 \times 10^{-12} & \sigma_{262} = 1.03 \times 10^{-12} \\ \sigma_{263} = 9.24 \times 10^{-13} & \sigma_{264} = 8.31 \times 10^{-13} \end{array}$$

If $\epsilon = 10^{-12}$, say, it might be reasonable to say that the numerical rank is approximately 260, but it is certainly not possible to specify it exactly.

The following exercises and theorem provide justification for the use of singular values to define the numerical rank of a matrix. We begin with an exercise that shows that a small perturbation in a matrix that is not of full rank can (and typically will) increase the rank. Here we refer to the exact rank, not the numerical rank.

Exercise 4.2.14 Let $A \in \mathbb{R}^{n \times m}$ with $\text{rank } r < \min\{n, m\}$. Use the SVD of A to show that for every $\epsilon > 0$ (no matter how small), there exists a full-rank matrix $A_\epsilon \in \mathbb{R}^{n \times m}$ such that $\|A - A_\epsilon\|_2 < \epsilon$. $\square$

The nonnegative number $\|A - A_\epsilon\|_2$ is a measure of the distance between the matrices A and A_ϵ . Exercise 4.2.14 shows that every rank-deficient matrix has full-rank matrices arbitrarily close to it. This suggests that matrices of full rank are abundant. This impression is strengthened by the next theorem and its corollary.

Theorem 4.2.15 Let $A \in \mathbb{R}^{n \times m}$ with $\text{rank}(A) = r > 0$. Let $A = U\Sigma V^T$ be the SVD of A , with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$. For $k = 1, \dots, r-1$, define $A_k = U\Sigma_k V^T$, where $\Sigma_k \in \mathbb{R}^{n \times m}$ is the diagonal matrix $\text{diag}\{\sigma_1, \dots, \sigma_k, 0, \dots, 0\}$. Then $\text{rank}(A_k) = k$, and

$$\sigma_{k+1} = \|A - A_k\|_2 = \min \{\|A - B\|_2 \mid \text{rank}(B) \leq k\}.$$

That is, of all matrices of rank k or less, A_k is closest to A .

Proof. It is obvious that $\text{rank}(A_k) = k$. Since $A - A_k = U(\Sigma - \Sigma_k)V^T$, it is clear that the largest singular value of $A - A_k$ is σ_{k+1} . Therefore $\|A - A_k\|_2 = \sigma_{k+1}$. It remains to be shown only that for any other matrix B of rank k or less, $\|A - B\|_2 \geq \sigma_{k+1}$.

Given any such B , note first that $\mathcal{N}(B)$ has dimension at least $m - k$, for $\dim(\mathcal{N}(B)) = m - \dim(\mathcal{R}(B)) = m - \text{rank}(B) \geq m - k$ by (4.1.9). Also, the space $\text{span}\{v_1, \dots, v_{k+1}\}$ has dimension $k + 1$. (As usual, $v_1, \dots, v_m$ denote the columns of V .) Since $\mathcal{N}(B)$ and $\text{span}\{v_1, \dots, v_{k+1}\}$ are two subspaces of $\mathbb{R}^m$, the sum of whose dimensions exceeds m , they must have a nontrivial intersection. Let $\hat{x}$ be a nonzero vector in $\mathcal{N}(B) \cap \text{span}\{v_1, \dots, v_{k+1}\}$. We can and will assume that $\|\hat{x}\|_2 = 1$. Since $\hat{x} \in \text{span}\{v_1, \dots, v_{k+1}\}$, there exist scalars $c_1, \dots, c_{k+1}$ such that $\hat{x} = c_1 v_1 + \dots + c_{k+1} v_{k+1}$. Because $v_1, \dots, v_{k+1}$ are orthonormal, $|c_1|^2 + \dots + |c_{k+1}|^2 = \|\hat{x}\|_2^2 = 1$. Since $\hat{x} \in \mathcal{N}(B)$, $B\hat{x} = 0$. Thus

$$(A - B)\hat{x} = A\hat{x} = \sum_{i=1}^{k+1} c_i A v_i = \sum_{i=1}^{k+1} \sigma_i c_i u_i.$$

Since $u_1, \dots, u_{k+1}$ are also orthonormal,

$$\|(A - B)\hat{x}\|_2^2 = \sum_{i=1}^{k+1} |\sigma_i c_i|^2 \geq \sigma_{k+1}^2 \sum_{i=1}^{k+1} |c_i|^2 = \sigma_{k+1}^2.$$

Therefore

$$\|A - B\|_2 \geq \frac{\|(A - B)\hat{x}\|_2}{\|\hat{x}\|_2} \geq \sigma_{k+1}.$$

□

Corollary 4.2.16 Suppose $A \in \mathbb{R}^{n \times m}$ has full rank. Thus $\text{rank}(A) = r$, where $r = \min\{n, m\}$. Let $\sigma_1 \geq \dots \geq \sigma_r$ be the singular values of A . Let $B \in \mathbb{R}^{n \times m}$ satisfy $\|A - B\|_2 < \sigma_r$. Then B also has full rank.

Exercise 4.2.17 Deduce Corollary 4.2.16 from Theorem 4.2.15. □

Exercise 4.2.18 In the first part of the proof of Theorem 4.2.15, we used the SVD of A , A_k , and $A - A_k$ in the form given by Theorem 4.1.1. Write down the other forms of the SVD of A_k and $A - A_k$: (a) Diagram (4.1.7), (b) Theorem 4.1.10, (c) Theorem 4.1.12. □

From Corollary 4.2.16 we see that if A has full rank, then all matrices sufficiently close to A also have full rank. From Exercise 4.2.14 we know that every rank-deficient matrix has full-rank matrices arbitrarily close to it. By Corollary 4.2.16, each of these full-rank matrices is surrounded by other matrices of full rank. In topological language, the set of matrices of full rank is an open, dense subset of $\mathbb{R}^{n \times m}$. Its complement, the set of rank-deficient matrices, is therefore closed and nowhere dense. Thus, in a certain sense, almost all matrices have full rank.

If a matrix does not have full rank, any small perturbation is almost certain to transform it to a matrix that does have full rank. It follows that in the presence of uncertainty in the data, it is impossible to calculate the (exact, theoretical) rank of a matrix or even detect that it is rank deficient. This is a generalization of the assertion,

made in Chapters 1 and 2, that it is (usually) impossible to determine whether a square matrix is singular.

Nevertheless, it is reasonable to call a matrix *numerically rank deficient* if it is very close to a rank-deficient matrix, since it could have been rank deficient except for a small perturbation, as was the case for the perturbed version of (4.2.13). Let ϵ be some positive number that represents the magnitude of the data uncertainties in the matrix A . If there exist matrices B of rank k such that $\|A - B\|_2 < \epsilon$ and, on the other hand, for every matrix C of rank $\leq k - 1$ we have $\|A - C\|_2 \gg \epsilon$, then it makes sense to say that the numerical rank of A is k . From Theorem 4.2.15 we know that this condition is satisfied if and only if

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k \gg \epsilon \geq \sigma_{k+1} \geq \cdots$$

This justifies the use of singular values to determine numerical rank.

Exercise 4.2.19 Use MATLAB to generate a random matrix. For example, the command `A = randn(40, 17);` produces a 40×17 matrix of independent identically distributed random numbers with mean zero and variance 1. Use MATLAB's `svd` command to determine the singular values of A . Notice that (unless you are really (un)lucky), all of the singular values are large. Repeat the experiment several times. Try matrices of various sizes. This shows that a matrix chosen at random (nearly always) has full rank. $\square$

Exercise 4.2.20 Use MATLAB to generate a random 8×6 matrix with rank 4. For example, you can proceed as follows.

```
A = randn(8, 4);
A(:, 5:6) = A(:, 1:2) + A(:, 3:4);
[Q, R] = qr(randn(6));
A = A*Q;
```

The first command produces a random 8×4 matrix (of rank 4). The second command adjoins two columns to A by adding columns 1 and 2 to columns 3 and 4, respectively. Now A is 8×6 and still has (numerical) rank 4. The third line generates a 6×6 random orthogonal matrix Q by performing a QR decomposition of a random 6×6 matrix. The last line recombines the columns of A by multiplying on the right by Q . The resulting matrix has numerical rank 4.

- Print out A on your computer screen. Can you tell by looking at it that it has (numerical) rank 4?
- Use MATLAB's `svd` command to obtain the singular values of A . How many are "large?" How many are "tiny?" (Use the command `format short e` to get a more accurate view of the singular values. Type `help format` for more information on the various output formats that are available.)
- Use MATLAB's `rank` command to confirm that the numerical rank is 4.
- Use the `rank` command with a low enough threshold that it returns the value 6. (Type `help rank` for information about how to do this.)

□

Exercise 4.2.21 The Kahan matrix $R_n(\theta)$ is an $n \times n$ upper triangular matrix depending on a parameter θ . Let $c = \cos \theta$ and $s = \sin \theta$. Then

$$R_n(\theta) = \begin{bmatrix} 1 & & & & \\ & s & & & \\ & & s^2 & & \\ & & & \ddots & \\ & & & & s^{n-1} \end{bmatrix} \begin{bmatrix} 1 & -c & -c & \cdots & -c \\ & 1 & -c & \cdots & -c \\ & & 1 & & -c \\ & & & \ddots & \vdots \\ & & & & 1 \end{bmatrix}.$$

If θ and n are chosen so that s is close to 1 and n is modestly large, then none of the main diagonal entries is extremely small. It thus appears that the matrix is far from rank deficient, but in this case appearances are deceiving. Consider $A = R_n(\theta)$ when $n = 90$ and $\theta = 1.2$ radians.

- Show that the largest main-diagonal entry of A is 1 and the smallest is greater than .001.
- To generate A in MATLAB and find its singular values, type

```
A = gallery('kahan',90,1.2,0);
sig = svd(A);
```

Examine σ_1, σ_{89} , and σ_{90} (It's not zero; try `format short e`). What is the numerical rank of A ? Type `rank(A)` to get MATLAB's opinion.

- Type `A = gallery('kahan',90,1.2,25)`; to get a slightly perturbed version of the Kahan matrix. (Type `help private/kahan` for more details.) Repeat part (b) for this perturbed matrix. Then get a QR decomposition with column pivoting: `[Q,R,E] = qr(A);`. Note that permutation matrix E is the identity matrix: `dif = norm(eye(90)-E)`, so no pivoting was done. Examine $R(90,90)$, and deduce that the QR decomposition with column pivoting failed to detect the numerical rank deficiency of A .

The Kahan matrix is just one of many matrices in Higham's gallery of test matrices for MATLAB. Type `help gallery` for a complete list. □

Orthogonal Decompositions

The QR decomposition with column pivoting gives $AE = QR$ or equivalently $A = QRE^T$, where E is a permutation matrix, a special type of orthogonal matrix. The SVD gives $A = U\Sigma V^T$. Both are examples of orthogonal decompositions $A = YTZ^T$, where Y and Z are orthogonal, and T has a simple form. The QR decomposition is much cheaper to compute than the SVD. However, the SVD always reveals the numerical rank of the matrix, whereas the QR decomposition may sometimes fail to do so, as we have seen in Exercise 4.2.21. Therefore there has

been considerable interest in producing an orthogonal decomposition that is cheap to compute yet reveals the rank of the matrix. Particularly noteworthy are the ULV and URV decompositions, which have the attractive feature that they can be updated inexpensively if A is modified by a matrix of rank one, as happens in signal processing applications. See [33, §12.5.5] for details.

Distance to Nearest Singular Matrix

We conclude this section by considering the implications of Theorem 4.2.15 for square, nonsingular matrices. Let $A \in \mathbb{R}^{n \times n}$ be nonsingular, and let A_s denote the singular matrix that is closest to A , in the sense that $\|A - A_s\|_2$ is as small as possible. In Theorem 2.3.1 we showed that

$$\frac{\|A - A_s\|}{\|A\|} \geq \frac{1}{\kappa(A)}$$

for any induced matrix norm, and we mentioned that for the 2-norm, equality holds. We now have the tools to prove this.

Corollary 4.2.22 *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. (Thus A has singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n > 0$.) Let A_s be the singular matrix that is closest to A , in the sense that $\|A - A_s\|_2$ is as small as possible. Then $\|A - A_s\| = \sigma_n$, and*

$$\frac{\|A - A_s\|_2}{\|A\|_2} = \frac{1}{\kappa_2(A)}.$$

These results are immediate consequences of Theorems 4.2.1, 4.2.4, and 4.2.15. In words, the distance from A to the nearest singular matrix is equal to the smallest singular value of A , and the “relative distance” to the nearest singular matrix is equal to the reciprocal of the condition number.

Additional Exercises

Exercise 4.2.23 Formulate and prove an expression for the Frobenius condition number $\kappa_F(A)$ for nonsingular $A \in \mathbb{R}^{n \times n}$ in terms of the singular values of A . $\square$

Exercise 4.2.24 Let $A \in \mathbb{R}^{n \times m}$ with singular values $\sigma_1 \geq \cdots \geq \sigma_m$ and right singular vectors $v_1, \dots, v_m$. We have seen that $\|Ax\|_2/\|x\|_2$ is maximized when $x = v_1$ and minimized when $x = v_m$. More generally show that for $k = 1, \dots, m$

$$\begin{aligned} \sigma_k &= \frac{\|Av_k\|_2}{\|v_k\|_2} = \max \left\{ \frac{\|Ax\|_2}{\|x\|_2} \mid x \neq 0, x \in \text{span}\{v_1, \dots, v_{k-1}\}^\perp \right\} \\ &= \min \left\{ \frac{\|Ax\|_2}{\|x\|_2} \mid x \neq 0, x \in \text{span}\{v_{k+1}, \dots, v_m\}^\perp \right\}. \end{aligned}$$

$\square$

Exercise 4.2.25

- (a) Let $A \in \mathbb{R}^{2 \times 2}$ with singular values $\sigma_1 \geq \sigma_2 > 0$. Show that the set $\{Ax \mid \|x\|_2 = 1\}$ (the image of the unit circle) is an ellipse in $\mathbb{R}^2$ whose major and minor semiaxes have lengths σ_1 and σ_2 , respectively.
- (b) Let $A \in \mathbb{R}^{n \times m}$, $n \geq m$, $\text{rank}(A) = m$. Show that the set $\{Ax \mid \|x\|_2 = 1\}$ is an m -dimensional hyperellipsoid with semiaxes $\sigma_1, \sigma_2, \dots, \sigma_m$. Notice that the lengths of the longest and shortest semiaxes are $\text{maxmag}(A)$ and $\text{minmag}(A)$, respectively.

□

4.3 THE SVD AND THE LEAST SQUARES PROBLEM

Let $A \in \mathbb{R}^{n \times m}$, $r = \text{rank}(A)$, and $b \in \mathbb{R}^n$, and consider the system of equations

$$Ax = b \quad (4.3.1)$$

with unknown $x \in \mathbb{R}^m$. If $n > m$, then the system is overdetermined, and we cannot expect to find an exact solution. Thus we will seek an x such that $\|b - Ax\|_2$ is minimized. This is exactly the least squares problem, which we studied in Chapter 3. There we found that if $n \geq m$ and $\text{rank}(A) = m$, the least squares problem has a unique solution. If $\text{rank}(A) < m$, the solution is not unique; there are many x for which $\|b - Ax\|_2$ is minimized. Even if $n < m$, it can happen that (4.3.1) does not have an exact solution, so we include that case as well. Thus we will make no assumption about the relative sizes of n and m .

Because the solution of the least squares problem is sometimes not unique, we will consider the following additional problem: of all the $x \in \mathbb{R}^m$ that minimize $\|b - Ax\|_2$, find the one for which $\|x\|_2$ is minimized. As we shall see, this problem always has a unique solution.

Initially we shall assume A and b are known exactly, and all computations are carried out exactly. Once we have settled the theoretical issues, we will discuss the practical questions.

Suppose we have the exact SVD $A = U\Sigma V^T$, where $U \in \mathbb{R}^{n \times n}$ and $V \in \mathbb{R}^{m \times m}$ are orthogonal, and

$$\Sigma = \begin{bmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{bmatrix}, \quad \hat{\Sigma} = \text{diag}\{\sigma_1, \dots, \sigma_r\},$$

with $\sigma_1 \geq \dots \geq \sigma_r > 0$. Because U is orthogonal,

$$\|b - Ax\|_2 = \|U^T(b - Ax)\|_2 = \|U^Tb - \Sigma(V^Tx)\|_2.$$

Letting $c = U^Tb$ and $y = V^Tx$, we have

$$\|b - Ax\|_2^2 = \|c - \Sigma y\|_2^2 = \sum_{i=1}^r |c_i - \sigma_i y_i|^2 + \sum_{i=r+1}^n |c_i|^2. \quad (4.3.2)$$

It is clear that this expression is minimized when and only when

$$y_i = \frac{c_i}{\sigma_i}, \quad i = 1, \dots, r.$$

Notice that when $r < m$, $y_{r+1}, \dots, y_m$ do not appear in (4.3.2). thus they have no effect on the residual and can be chosen arbitrarily. Among all the solutions so obtained, $\|y\|_2$ is clearly minimized when and only when $y_{r+1} = \dots = y_m = 0$. Since $x = Vy$ and V is orthogonal, $\|x\|_2 = \|y\|_2$. Thus $\|x\|_2$ is minimized when and only when $\|y\|_2$ is. This proves that the least squares problem has exactly one minimum norm solution.

It is useful to repeat the development using partitioned matrices. Let

$$c = \begin{bmatrix} \hat{c} \\ d \end{bmatrix} \quad \text{and} \quad y = \begin{bmatrix} \hat{y} \\ z \end{bmatrix},$$

where $\hat{c}, \hat{r} \in \mathbb{R}^r$. Then

$$c - \Sigma y = \begin{bmatrix} \hat{c} \\ d \end{bmatrix} - \begin{bmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \hat{y} \\ z \end{bmatrix} = \begin{bmatrix} \hat{c} - \hat{\Sigma} \hat{y} \\ d \end{bmatrix},$$

so

$$\|b - Ax\|_2^2 = \|c - \Sigma y\|_2^2 = \|\hat{c} - \hat{\Sigma} \hat{y}\|_2^2 + \|d\|_2^2.$$

This is minimized when and only when $\hat{y} = \hat{\Sigma}^{-1} \hat{c}$, that is, $y_i = c_i / \sigma_i$, $i = 1, \dots, r$. We can choose z arbitrarily, but we get the minimum norm solution by taking $z = 0$. The norm of the minimal residual is $\|d\|_2$. This solves the problem completely in principle. We summarize the procedure:

1. Calculate $\begin{bmatrix} \hat{c} \\ d \end{bmatrix} = c = U^T b$.
 2. Let $\hat{y} = \hat{\Sigma}^{-1} \hat{c}$.
 3. If $r < m$, choose $z \in \mathbb{R}^{m-r}$ arbitrarily.
(For minimum-norm solution, take $z = 0$.)
 4. Let $y = \begin{bmatrix} \hat{y} \\ z \end{bmatrix} \in \mathbb{R}^m$.
 5. Let $x = Vy$.
- (4.3.3)

Practical Considerations

In practice we do not know the exact rank of A . It is best to use the numerical rank, which we discussed in Section 4.2. All “tiny” singular values should be set to zero.

We have solved the least squares problem under the assumption that we have the matrices U and V at hand. However, you can easily check that the calculation of

$\hat{c}$ uses only the first r columns of U , where, in practice, r is the numerical rank. If only the minimum-norm solution is wanted, only the first r columns of V are needed. While the numerical rank is usually not known in advance, it can never exceed $\min\{n, m\}$, so at most $\min\{n, m\}$ columns of U and V are needed.

If $n \gg m$, the computation of U can be expensive, even if we only compute the first m columns. In fact the computation of U can be avoided completely. U is the product of many reflectors and rotators that are generated in the computation of the SVD (discussed in Section 5.9). Since U is needed only so that we can compute $c = U^T b$, we can simply update b instead of assembling U . As each rotator or reflector U_i is generated, we make the update $b \leftarrow U_i^T b$. In the end, b will have been transformed into c . This is much less expensive than computing U explicitly just to get $c = U^T b$. In the process, we get not only $\hat{c}$, but also d , from which we can compute the residual $\|d\|_2$ inexpensively. If several least squares problems with the same A but different right-hand sides $b^{(1)}, b^{(2)}, \dots$ are to be solved, the updates must be applied to all of the $b^{(j)}$ at once, since the U_i will not be saved.

No matter how the calculations are organized, the SVD is an expensive way to solve the least squares problem. Its principal advantage is that it gives a completely reliable means of determining the numerical rank for rank-deficient least squares problems.

The Pseudoinverse

The pseudoinverse, also known as the Moore-Penrose generalized inverse, is an interesting generalization of the ordinary inverse. Although only square nonsingular matrices have inverses in the ordinary sense, every $A \in \mathbb{R}^{n \times m}$ has a pseudoinverse. Just as the solution of a square nonsingular system $Ax = b$ can be expressed in terms of A^{-1} as $x = A^{-1}b$, the minimum-norm solution to a least squares problem with coefficient matrix $A \in \mathbb{R}^{n \times m}$ can be expressed in terms of the pseudoinverse $A^\dagger$ as $x = A^\dagger b$.

Given $A \in \mathbb{R}^{n \times m}$ with rank r , the action of A is completely described by the diagram

$$\begin{array}{ccc}
 & A & \\
 v_1 & \xrightarrow{\sigma_1} & u_1 \\
 v_2 & \xrightarrow{\sigma_2} & u_2 \\
 \vdots & & \vdots \\
 v_r & \xrightarrow{\sigma_r} & u_r \\
 \left. \begin{array}{l} v_{r+1} \\ \vdots \\ v_m \end{array} \right\} & \longrightarrow & 0,
 \end{array}$$

where $v_1, \dots, v_m$ and $u_1, \dots, u_n$ are complete orthonormal sets of right and left singular vectors, respectively, and $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ are the nonzero singular values of A . In matrix form,

$$A = U \Sigma V^T.$$

We wish to define the pseudoinverse $A^\dagger \in \mathbb{R}^{m \times n}$ so that it is as much like a true inverse as possible. Therefore we must certainly require $A^\dagger u_i = \sigma_i^{-1} v_i$ for $i = 1, \dots, r$. A reasonable choice for $A^\dagger u_{r+1}, \dots, A^\dagger u_n$ is to make them zero. Thus we define the *pseudoinverse* of A to be the matrix $A^\dagger \in \mathbb{R}^{m \times n}$ that is uniquely specified by the diagram

$$\begin{array}{ccc}
 & A^\dagger & \\
 u_1 & \xrightarrow{\sigma_1^{-1}} & v_1 \\
 u_2 & \xrightarrow{\sigma_2^{-1}} & v_2 \\
 \vdots & & \vdots \\
 u_r & \xrightarrow{\sigma_r^{-1}} & v_r \\
 \left. \begin{array}{l} u_{r+1} \\ \vdots \\ u_n \end{array} \right\} & \longrightarrow & 0.
 \end{array}$$

We see immediately that $\text{rank}(A^\dagger) = \text{rank}(A)$, $u_1, \dots, u_n$ and $v_1, \dots, v_m$ are right and left singular vectors of $A^\dagger$, respectively, and $\sigma_1^{-1}, \dots, \sigma_r^{-1}$ are the nonzero singular values. The restricted operators $A : \text{span}\{v_1, \dots, v_r\} \rightarrow \text{span}\{u_1, \dots, u_r\}$ and $A^\dagger : \text{span}\{u_1, \dots, u_r\} \rightarrow \text{span}\{v_1, \dots, v_r\}$ are true inverses of one another.

What does $A^\dagger$ look like as a matrix? You can answer this question in the simplest case by working the following exercise.

Exercise 4.3.4 Show that if

$$\Sigma = \begin{bmatrix} \hat{\Sigma} & 0 \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{n \times m}, \quad \hat{\Sigma} = \text{diag}\{\sigma_1, \dots, \sigma_r\},$$

then

$$\Sigma^\dagger = \begin{bmatrix} \hat{\Sigma}^{-1} & 0 \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

□

To see what $A^\dagger$ looks like in general, simply note that the equations

$$A^\dagger u_i = \begin{cases} v_i \sigma_i^{-1} & i = 1, \dots, r \\ 0 & i = r+1, \dots, n \end{cases}$$

can be expressed as a single matrix equation $A^\dagger U = V \Sigma^\dagger$, where $\Sigma^\dagger$ is as given in Exercise 4.3.4. Thus, since U is orthogonal,

$$A^\dagger = V \Sigma^\dagger U^T. \quad (4.3.5)$$

This is the SVD of $A^\dagger$ in matrix form. If we let $\hat{U}$ and $\hat{V}$ denote the first r columns of U and V , respectively, we can rewrite (4.3.5) in the condensed form

$$A^\dagger = \hat{V} \hat{\Sigma}^{-1} \hat{U}^T, \quad (4.3.6)$$

as in Theorem 4.1.10. Equation (4.3.6) gives us a means of calculating $A^\dagger$ by computing the SVD of A . However, there is seldom any reason to compute the pseudoinverse; it is mainly a theoretical tool. In this respect the pseudoinverse plays a role much like that of the ordinary inverse.

It is easy to make the claimed connection between the pseudoinverse and the least squares problem.

Theorem 4.3.7 Let $A \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^n$, and let $x \in \mathbb{R}^m$ be the minimum-norm solution of

$$\|b - Ax\|_2 = \min_{w \in \mathbb{R}^m} \|b - Aw\|_2.$$

Then $x = A^\dagger b$.

Proof. By (4.3.3),

$$\begin{aligned} x &= Vy = V \begin{bmatrix} \hat{y} \\ 0 \end{bmatrix} = V \begin{bmatrix} \hat{\Sigma}^{-1} \hat{c} \\ 0 \end{bmatrix} = V \begin{bmatrix} \hat{\Sigma}^{-1} & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \hat{c} \\ d \end{bmatrix} \\ &= V \Sigma^\dagger c = V \Sigma^\dagger U^T b = A^\dagger b. \end{aligned}$$

□

The pseudoinverse is used in the study of the sensitivity of the rank-deficient least squares problem. See [7].

Exercise 4.3.8 Show that if $A = \sum_{i=1}^r \sigma_i u_i v_i^T$, then $A^\dagger = \sum_{i=1}^r \sigma_i^{-1} v_i u_i^T$. □

Exercise 4.3.9 Work this exercise using pencil and paper. You might like to use MATLAB to check your work. Let

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

- Find the SVD of A . You may use the condensed form given by Theorem 4.1.10 or Theorem 4.1.12.
- Calculate $A^\dagger$.
- Calculate the minimum norm solution of the least-squares problem for the overdetermined system $Ax = b$.
- Find a basis for $\mathcal{N}(A)$.
- Find all solutions of the least-squares problem.

□

Exercise 4.3.10 MATLAB's `pinv` command returns the pseudoinverse of a matrix. Use the `pinv` command to find the pseudoinverse of

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}.$$

Use MATLAB's `svd` command to find the singular values of $AA^\dagger$ and $A^\dagger A$. Explain why these results are what you would expect. $\square$

Exercise 4.3.11 (Pseudoinverses of full-rank matrices)

- Show that if $A \in \mathbb{R}^{n \times m}$, $n \geq m$, and $\text{rank}(A) = m$, then $A^\dagger = (A^T A)^{-1} A^T$ (cf. Theorem 4.2.11 and Exercise 4.2.12). Compare this result with the normal equations (3.5.22).
- Show that if $A \in \mathbb{R}^{n \times m}$, $n \leq m$, and $\text{rank}(A) = n$, then $A^\dagger = A^T (A^T A)^{-1}$.

$\square$

Additional Exercises

Exercise 4.3.12 Study the effects of treating very small nonzero singular values as nonzeros in the solution of the least squares problem. $\square$

Exercise 4.3.13 Suppose $A \in \mathbb{R}^{n \times m}$, $n < m$, and $\text{rank}(A) = n$.

- Show that in this case the minimum-norm least squares problem is actually a constrained minimization problem.
- This problem can be solved by an SVD. Show that it can also be solved by an LQ decomposition: $A = LQ$, where $L = [\tilde{L} \ 0] \in \mathbb{R}^{n \times m}$, $\tilde{L} \in \mathbb{R}^{n \times n}$ is nonsingular and lower triangular, and $Q \in \mathbb{R}^{m \times m}$ is orthogonal.
- How does one calculate an LQ decomposition? Sketch an algorithm for calculating the LQ decomposition and solving the constrained minimization problem.

$\square$

Exercise 4.3.14

- Let $A \in \mathbb{R}^{n \times m}$ and $B = A^\dagger \in \mathbb{R}^{m \times n}$. Show that the following four relationships hold:

$$\begin{aligned} BAB &= B & ABA &= A \\ (BA)^T &= BA & (AB)^T &= AB. \end{aligned} \quad (4.3.15)$$

- Conversely, show that if A and B satisfy (4.3.15), then $B = A^\dagger$. Thus equations (4.3.15) characterize the pseudoinverse. In many books these equations are used to define the pseudoinverse.

$\square$

4.4 SENSITIVITY OF THE LEAST SQUARES PROBLEM

In this section we discuss the sensitivity of the solution of the least squares problem under perturbations of A and b . For simplicity we consider only the full-rank case with $n \geq m$. See [7] for more general results.

Given $A \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^n$, with $n \geq m$ and $\text{rank}(A) = m$, there is a unique $x \in \mathbb{R}^m$ such that

$$\|b - Ax\|_2 = \min_{w \in \mathbb{R}^m} \|b - Aw\|_2.$$

If we now perturb A and b slightly, the solution x will be altered. We will consider the question of how sensitive x is to perturbations in A and b . We will also consider the sensitivity of the residual $r = b - Ax$. The sensitivity analysis can be combined with backward error analyses to provide an assessment of the accuracy of various methods for solving the least squares problem.

From Section 3.5 it is clear that we can think of the solution of the least squares problem as a two-stage process. First we find a $y \in \mathcal{R}(A)$ whose distance from b is minimal:

$$\|b - y\|_2 = \min_{s \in \mathcal{R}(A)} \|b - s\|_2.$$

Then the least squares solution $x \in \mathbb{R}^m$ is found by solving the equation $Ax = y$ exactly. Because A has full rank, the solution is unique. Even though $A \in \mathbb{R}^{n \times m}$ and $y \in \mathbb{R}^n$, the matrix equation $Ax = y$ is effectively a system of m equations in m unknowns, for y lies in the m -dimensional subspace $\mathcal{R}(A)$, and $A : \mathbb{R}^m \rightarrow \mathcal{R}(A)$ can be viewed as a mapping of one m -dimensional space into another.

Exercise 4.4.1 The m -dimensional nature of the system can be revealed by a judicious change of coordinate system.

- Show that the QR decomposition does just this. How is the vector y determined? How is the system $Ax = y$ solved?
- Repeat part (a) using the SVD in place of the QR decomposition.

□

Let $r = b - y = b - Ax$. This is the residual of the least squares problem. By Theorem 3.5.15, r is orthogonal to $\mathcal{R}(A)$. In particular, b , y , and r form a right triangle, as illustrated in Figure 4.1. The angle θ between b and y is the smallest

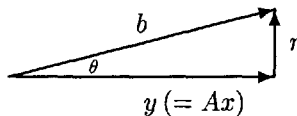


Fig. 4.1 Right triangle formed by b , y , and r

angle between b and an element of $\mathcal{R}(A)$. It is clear that

$$\|r\|_2 = \|b\|_2 \sin \theta, \quad (4.4.2)$$

$$\|y\|_2 = \|b\|_2 \cos \theta. \quad (4.4.3)$$

These equations and Figure 4.1 will prove useful in our sensitivity analysis.

The Effect of Perturbations of b

At first we will examine the effects of perturbations of b only. Given a perturbation δb , let $y + \delta y$ denote the element of $\mathcal{R}(A)$ that is closest to $b + \delta b$, and let $\hat{x} = x + \delta x$ be the exact solution of $A\hat{x} = y + \delta y$. Then $\hat{x}$ is the minimizer of $\|(b + \delta b) - Ax\|_2$. We would like to be able to say that if $\|\delta b\|_2/\|b\|_2$ is small, then $\|\delta x\|_2/\|x\|_2$ is also small. As we shall see, there are two reasons why this might fail to be the case. The first is that $\|\delta y\|_2/\|y\|_2$ might fail to be small. To see this, refer to Figure 4.1 and consider what happens when b is orthogonal or nearly orthogonal to $\mathcal{R}(A)$. Then y is either zero or very small. It is clear that a small perturbation of b in a direction parallel to $\mathcal{R}(A)$ will cause a perturbation δy that is large relative to $\|y\|_2$. The second problem is that even if $\|\delta y\|_2/\|y\|_2$ is small, $\|\delta x\|_2/\|x\|_2$ can be large if the linear system $Ax = y$ is ill conditioned. Any error bound for $\|\delta x\|_2/\|x\|_2$ must reflect both of these factors. In the next exercise you will derive such a bound.

Exercise 4.4.4

(a) Defining $\delta r = \delta b - \delta y$, we have $\delta b = \delta y + \delta r$. Show that δr is orthogonal to y . Deduce that $\|\delta y\|_2 \leq \|\delta b\|_2$.

(b) Show that, provided $\cos \theta \neq 0$,

$$\frac{\|\delta y\|_2}{\|y\|_2} \leq \frac{1}{\cos \theta} \frac{\|\delta b\|_2}{\|b\|_2}. \quad (4.4.5)$$

(c) Show that $\|y\|_2 \leq \text{maxmag}(A)\|x\|_2$, $\|\delta y\|_2 \geq \text{minmag}(A)\|\delta x\|_2$, and

$$\frac{\|\delta x\|_2}{\|x\|_2} \leq \kappa_2(A) \frac{\|\delta y\|_2}{\|y\|_2}. \quad (4.4.6)$$

(d) Conclude that

$$\frac{\|\delta x\|_2}{\|x\|_2} \leq \frac{\kappa_2(A)}{\cos \theta} \frac{\|\delta b\|_2}{\|b\|_2}. \quad (4.4.7)$$

□

Inequality (4.4.5) shows that if θ is not close to $\pi/2$, that is, if b is not nearly orthogonal to the range of A , a small perturbation in b results in a small perturbation in y . Of course, if θ is close to $\pi/2$, we can have a disaster. But this almost never happens in real problems; the angle θ , like $\|r\|_2$, is a measure of how well the least

squares solution fits the data. It will not be close to $\pi/2$ unless all of the vectors in $\mathcal{R}(A)$ approximate b very poorly. Thus, as long as the problem is formulated so that the least squares solution fits the data reasonably well, (4.4.5) will give a useful bound on $\|\delta y\|_2/\|y\|_2$.

Inequality (4.4.6) shows the effect of $\kappa_2(A)$. This is really no different from the inequality (2.2.5), which we derived during the discussion of the sensitivity of square linear systems.

Inequality (4.4.7) combines the two effects. We see that if $\kappa_2(A)$ is not large, and $\cos \theta$ is not close to zero, then a small perturbation in b results in a small perturbation in x .

Inequality (4.4.7) can also be derived using the normal equations (3.5.22). We will work through this second derivation as preparation for the discussion of the effects of perturbations of A . Here we will make use of some of our elementary results on singular values.

From Theorem 3.5.21 we know that the solution of the least squares problem satisfies the normal equations

$$A^T A x = A^T b. \quad (4.4.8)$$

The solution of the perturbed problem satisfies the perturbed normal equations

$$A^T A(x + \delta x) = A^T(b + \delta b). \quad (4.4.9)$$

Subtracting (4.4.8) from (4.4.9), we find that $(A^T A)\delta x = A^T \delta b$, or

$$\delta x = (A^T A)^{-1} A^T \delta b.$$

Thus $\|\delta x\|_2 \leq \|(A^T A)^{-1} A^T\|_2 \|\delta b\|_2$. By Theorem 4.2.11, $\|(A^T A)^{-1} A^T\|_2 = \sigma_m^{-1}$, where σ_m is the smallest singular value of A , so

$$\|\delta x\|_2 \leq \sigma_m^{-1} \|\delta b\|_2. \quad (4.4.10)$$

On the other hand, we see from (4.4.3) that $\|b\|_2 \cos \theta = \|y\|_2 = \|Ax\|_2 \leq \|A\|_2 \|x\|_2 = \sigma_1 \|x\|_2$. Thus

$$\frac{1}{\|x\|_2} \leq \frac{\sigma_1}{\cos \theta} \frac{1}{\|b\|_2}. \quad (4.4.11)$$

Multiplying (4.4.10) by (4.4.11), and recalling that $\kappa_2(A) = \sigma_1/\sigma_m$ (Theorem 4.2.4), we get (4.4.7).

The Effect of Perturbations of A

Unfortunately perturbations of A have a more severe effect than perturbations of b . As the following Theorem shows, the sensitivity of x to perturbations of A depends, in part, on the square of the condition number of A . Our approach will be to state the theorem, discuss its implications, then prove it.

Theorem 4.4.12 Let $A \in \mathbb{R}^{n \times m}$, $b \in \mathbb{R}^n$, $n \geq m$, and $\text{rank}(A) = m$. Let $x \in \mathbb{R}^m$ be the unique solution of

$$\|b - Ax\| = \min_{w \in \mathbb{R}^m} \|b - Aw\|_2.$$

Let θ denote the angle between b and Ax , and assume $\theta \neq \pi/2$. Let $\delta A \in \mathbb{R}^{n \times m}$, $\delta b \in \mathbb{R}^n$, $\epsilon_A = \|\delta A\|_2 / \|A\|_2$, $\epsilon_b = \|\delta b\|_2 / \|b\|_2$, and $\epsilon = \max\{\epsilon_A, \epsilon_b\}$. Assume $\epsilon \ll 1$, and in particular, $\epsilon_A < 1/\kappa_2(A)$. Let $\hat{x} = x + \delta x \in \mathbb{R}^m$ be the unique solution of

$$\|(b + \delta b) - (A + \delta A)\hat{x}\| = \min_{w \in \mathbb{R}^m} \|(b + \delta b) - (A + \delta A)w\|_2.$$

Then

$$\frac{\|\delta x\|_2}{\|x\|_2} \leq \frac{2\kappa_2(A)}{\cos \theta} \epsilon_b + 2(\kappa_2(A)^2 \tan \theta + \kappa_2(A)) \epsilon_A + O(\epsilon^2). \quad (4.4.13)$$

Let $r = b - Ax$ and $\hat{r} = r + \delta r = (b + \delta b) - (A + \delta A)\hat{x}$. Then

$$\frac{\|\delta r\|_2}{\|b\|_2} \leq 2\epsilon_b + 3\kappa_2(A)\epsilon_A + O(\epsilon^2). \quad (4.4.14)$$

In both of the bounds the term $O(\epsilon^2)$ stands for terms that contain factors ϵ_A^2 or $\epsilon_A \epsilon_b$ and are negligible. If we take $\epsilon_A = 0$ in (4.4.13), we obtain (4.4.7), except for an inessential factor 2.

The most striking feature of (4.4.13) is that it depends on the square of $\kappa_2(A)$. This is not surprising if one considers that a perturbation in A causes a perturbation in $\mathcal{R}(A)$ that turns out to be (at worst) proportional to $\kappa_2(A)$. Thus a perturbation ϵ_A can cause a change of magnitude $\kappa_2(A)\epsilon_A$ in y ($= Ax \in \mathcal{R}(A)$), which can in turn cause a change of magnitude $\kappa_2(A)^2 \epsilon$ in x .

The presence of $\kappa_2(A)^2$ in (4.4.13) means that even if A is only mildly ill conditioned, a small perturbation in A can cause a large change in x . An exception is the class of problems for which the least squares solution fits the data exactly; that is, $r = 0$. Then $\tan \theta = 0$ as well, and the offending term disappears. While a perfect fit is unusual, a good fit is not. If the fit is good, $\tan \theta$ will be small and will partially cancel out the factor $\kappa_2(A)^2$.

Since we measure the size of residuals relative to b , it is reasonable to measure the change in the residual relative to $\|b\|_2$, as we have done in (4.4.14). Notice that (4.4.14) depends on $\kappa_2(A)$, not $\kappa_2(A)^2$. This means that the residual, and the goodness of fit, is generally much less sensitive to perturbations than the least squares solution is.

Keeping the condition number under control. From Theorem 4.4.12 it is clear that it is important to avoid having to solve least squares problems for which the coefficient matrix is ill conditioned. This is something over which you have some control. For the purpose of illustration, consider the problem of fitting a set

of data points by a polynomial of low degree. Then the system to be solved has the form (3.1.3); the coefficient matrix is determined by the point abscissae $t_1, t_2, \dots, t_n$ and the basis functions $\phi_1, \phi_2, \dots, \phi_m$. In principle any basis for the space of approximating polynomials can be used. However, the choice of basis affects the condition number of the system.

Example 4.4.15 Suppose we need to find the least squares first-degree polynomial for a set of seven points whose abscissae are

t_1	t_2	t_3	t_4	t_5	t_6	t_7
1.01	1.02	1.03	1.04	1.05	1.06	1.07

The obvious choice of basis for the first-degree polynomials is $\phi_1(t) = 1$ and $\phi_2(t) = t$. Let us contrast the behavior of this basis with that of the more carefully constructed basis $\tilde{\phi}_1 = 1$ and $\tilde{\phi}_2 = 30(t - 1.04)$. Using (3.1.3) we find that the two bases give rise to the coefficient matrices

$$A = \begin{bmatrix} 1 & 1.01 \\ 1 & 1.02 \\ 1 & 1.03 \\ 1 & 1.04 \\ 1 & 1.05 \\ 1 & 1.06 \\ 1 & 1.07 \end{bmatrix} \quad \tilde{A} = \begin{bmatrix} 1 & -.9 \\ 1 & -.6 \\ 1 & -.3 \\ 1 & 0 \\ 1 & .3 \\ 1 & .6 \\ 1 & .9 \end{bmatrix},$$

respectively. According to MATLAB, the condition numbers are $\kappa_2(A) \approx 104$ and $\kappa_2(\tilde{A}) \approx 1.67$. It is not hard to see why $\tilde{A}$ is so much better than A . The abscissae are concentrated in the interval $[1.01, 1.07]$, on which the function ϕ_2 varies very little; it looks a lot like ϕ_1 . Consequently the two columns of A are nearly equal, and A is (mildly) ill conditioned. By contrast the basis $\tilde{\phi}_1, \tilde{\phi}_2$ was chosen with the interval $[1.01, 1.07]$ in mind. $\tilde{\phi}_2$ is centered on the interval and varies from -0.9 to $+0.9$, while $\tilde{\phi}_1$ remains constant. As a consequence the columns of $\tilde{A}$ are not close to being linearly dependent. (In fact they are orthogonal.) The factor 30 in the definition of $\tilde{\phi}_2$ guarantees that the second column of $\tilde{A}$ has roughly the same magnitude as the first column, so the columns are not out of scale. Thus $\tilde{A}$ is well conditioned. $\square$

In general one wants to choose functions that are not close to being linearly dependent on the interval of interest. Then the columns of the resulting A will not be close to being linearly dependent. Consequently A will be well conditioned.

Accuracy of techniques for solving the least squares problem. A standard backward error analysis [41] (or Exercises 3.2.70–3.2.73) shows that the QR decomposition method using reflectors or rotators is normwise backward stable. Thus the computed solution is the exact minimizer of a perturbed function

$$\|(b + \delta b) - (A + \delta A)w\|_2,$$